

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



CO3084 - Mật mã học và Mã hóa Thông tin (TN)

Problem Set 3

Asymmetric Cryptography

SVTH: Nguyễn Tấn Tài - 2212990

TP. Hồ Chí Minh, Tháng 10/2025

Mục lục

1	Độ Khó Mật Mã và Các Tác Động Trong Thế Giới Thực	1
1.1	Câu 1. (10 điểm) Phá Vỡ Mật Mã: Các Kịch Bản Tấn Công	1
1.1.1	(a) (5 điểm) Kịch Bản "Ngày Tận Thế Của Mật Mã"	1
1.1.2	(b) (5 điểm) Vấn Đề "Thông Số Diffie-Hellman Yếu"	4
1.2	Câu 2. (10 điểm) Kiến Trúc Bảo Mật Dựa Trên Logarit Rời Rạc	9
1.2.1	(a) (5 điểm) Thảm Họa Do Tham Số Yếu	9
1.2.2	(b) (5 điểm) So Sánh Elliptic Curve và Finite Field	18
2	Diffie-Hellman Trong Môi Trường Thù địch	25
2.1	Câu 1. (10 điểm) Các Kịch Bản Tấn Công và Phòng Thủ	25
2.1.1	(a) (5 điểm) Kẻ Tấn Công Trung Gian Hoàn Hảo (Perfect Man-in-the-Middle)	25
2.1.2	(b) (5 điểm) Kịch Bản "Người Tố Giác Hoang Mang" (Paranoid Whistleblower)	26
2.2	Câu 2. (10 điểm) Thử Thách Thiết Kế Giao Thức	26
2.2.1	(a) (5 điểm)	26
2.2.2	(b) (5 điểm)	26
3	Kỹ Thuật Bảo Mật Đường Cong Elliptic (Elliptic Curve Security Engineering)	29
3.1	Câu 1. (15 điểm) Lựa Chọn Đường Cong Khi Chịu Áp Lực (Curve Selection Under Pressure)	29
3.1.1	(a) (8 điểm) Tranh Cãi Về "Backdoor" Của Chính Phủ	29
3.1.2	(b) (7 điểm) Kịch Bản Tấn Công "Invalid Curve"	31
3.2	Câu 2. (15 điểm) Phân Tích Lỗ Hổng Triển Khai (Implementation Vulnerability Analysis)	32
3.2.1	(a) (8 điểm) Phòng Thí Nghiệm Tấn Công Kênh Bên (Side-Channel Attack Laboratory)	32
3.2.2	(b) (7 điểm) Khủng Hoảng Xác Thực Ed25519 (Ed25519 Validation Crisis)	42
4	Case Studies Mật Mã Ứng Dụng (Applied Cryptography Case Studies)	43
4.1	Câu 1. (15 điểm) Phân Tích Cơ Chế Chữ Ký Cho Tiền Điện Tử	43
4.1.1	(a) (5 điểm) So Sánh ECDSA và Ed25519	43
4.1.2	(b) (5 điểm) Thảo Luận Về Hệ Quả Của Tính Có Thể Thay Đổi Chữ Ký (Signature Malleability)	43
4.1.3	(c) (5 điểm) Đưa Ra Khuyến Nghị Cuối Cùng	43
4.2	Câu 2. (15 điểm) Kiến Trúc Hệ Thống Liên Lạc An Toàn	44
4.2.1	(a) (5 điểm) Chỉ Định Các Lựa Chọn Thuật Toán Mật Mã	44
4.2.2	(b) (5 điểm) Mô Tả Kiến Trúc Quản Lý Khóa	45
4.2.3	(c) (5 điểm) Phân Tích Tính Bảo Mật Của Hệ Thống	45

1 Độ Khó Mật Mã và Các Tác Động Trong Thế Giới Thực

1.1 Câu 1. (10 điểm) Phá Vỡ Mật Mã: Các Kịch Bản Tấn Công

1.1.1 (a) (5 điểm) Kịch Bản "Ngày Tận Thế Của Mật Mã"

Hãy tưởng tượng bạn thức dậy vào ngày mai và thấy trên các bản tin dòng tít:

“Đột phá khoa học: P so với NP đã được giải quyết – Các nhà khoa học máy tính chứng minh rằng $P = NP$!”

i. Với tư cách là Giám đốc An ninh (Chief Security Officer) của một ngân hàng lớn, hãy viết một bản ghi nhớ khẩn cấp (crisis response memo) nêu rõ:

- Những hệ thống nào sẽ ngay lập tức bị sụp đổ,
- Và bạn sẽ triển khai các biện pháp khẩn cấp nào để đối phó tình huống đó.

ii. Hãy thiết kế một mô hình bảo mật thay thế cho ngân hàng trực tuyến trong một thế giới mà $P = NP$. Bạn sẽ dựa vào những giả định nào khác để bảo vệ an toàn cho hệ thống?

Trả lời:

i. Bản ghi nhớ khẩn cấp (Crisis Response Memo)

Độ mật - Khẩn cấp

Đến: Ban Lãnh Đạo, Toàn Bộ Phòng IT & An Ninh

Từ: Giám Đốc An Ninh (CSO)

Ngày: [Ngày mai]

Chủ đề: **Khẩn cấp - Sụp đổ toàn bộ hệ thống mật mã**

Tình trạng nguy cấp cấp độ 1

Chứng minh $P = NP$ đã được công bố. Toàn bộ nền tảng bảo mật của ngân hàng hiện không còn giá trị.

Các hệ thống sụp đổ ngay lập tức

1. Hệ thống mã hóa bất đối xứng - Sụp đổ hoàn toàn

- RSA (Factoring): Tất cả khóa RSA bị bẻ gãy
- Diffie-Hellman / ECDH (DLP): Trao đổi khóa không còn an toàn
- Chữ ký số (ECDSA, EdDSA): Không thể xác thực giao dịch

Hệ quả:

- Kẻ tấn công giải mã mọi giao dịch HTTPS/TLS
- Chữ ký số bị giả mạo
- Không thể xác thực danh tính khách hàng online

2. Các chức năng ngân hàng bị ảnh hưởng

- Internet Banking & Mobile Banking
- ATM Network
- SWIFT / Chuyển tiền quốc tế
- Thẻ tín dụng / Chip EMV

Biện pháp khẩn cấp

Giai đoạn 1: Giờ 0-2 (Phong tỏa)

1. Tạm dừng mọi giao dịch trực tuyến

- Đóng Internet Banking & Mobile Banking
- Tạm dừng giao dịch ATM từ xa
- Thông báo khách hàng qua SMS

2. Cách ly mạng quan trọng

- Ngắt kết nối internet với core banking system
- Chuyển sang mạng vật lý riêng biệt (air-gapped)

Giai đoạn 2: Giờ 2-24 (Chuyển đổi)

3. Quay về phương thức vật lý

- Mở cửa chi nhánh ngoài giờ
- Tăng nhân sự quầy giao dịch
- Sử dụng giấy tờ với chữ ký tay

4. Mã hóa đối xứng với khóa phân phối thủ công

- Gửi khóa AES qua courier an toàn đến chi nhánh
- Sử dụng két sắt, túi niêm phong

Giai đoạn 3: Tuần 1-4 (Xây dựng lại)

5. Triển khai Quantum Key Distribution (QKD)

- Phân phối khóa lượng tử (nếu có thiết bị)
- An toàn dựa trên vật lý, không bị $P=NP$ ảnh hưởng

6. Hợp tác với Ngân Hàng Trung Ương

- Phối hợp với các ngân hàng khác
- Yêu cầu hướng dẫn từ cơ quan quản lý

ii. Mô hình bảo mật thay thế trong thế giới $P = NP$

Nguyên tắc cơ bản

Trong thế giới $P = NP$, không thể dựa vào độ khó tính toán. Phải chuyển sang các giả định khác:

Mô hình 1: Bảo mật dựa vào vật lý

A. Quantum Key Distribution (QKD)

- **Nguyên lý:** Sử dụng photon lượng tử để phân phối khóa
- **Lý do an toàn:** Dựa trên nguyên lý bất định Heisenberg - kẻ nghe lén làm thay đổi trạng thái lượng tử và bị phát hiện
- **Ứng dụng:** Kết nối giữa chi nhánh chính với data center

B. One-Time Pad (OTP)

- **Nguyên lý:** Mã hóa hoàn hảo theo Shannon (1949)
- **Yêu cầu:** Khóa dài bằng bản rõ, ngẫu nhiên hoàn toàn, chỉ dùng 1 lần
- **Ứng dụng:** Phân phối USB chứa khóa cho khách hàng VIP hàng tháng

Mô hình 2: Bảo mật dựa vào điều kiện vật lý

C. Air-Gapped Networks

- **Nguyên lý:** Hệ thống không kết nối internet
- **Giả định:** Kẻ tấn công không có quyền truy cập vật lý
- **Ứng dụng:** Core banking chạy trên mạng riêng

D. Multi-Party Computation (MPC)

- **Nguyên lý:** Phân tán tính toán qua nhiều bên
- **Giả định:** Ít nhất k trong n bên là trung thực
- **Ứng dụng:** Giao dịch lớn cần xác nhận từ 3/5 node độc lập

Mô hình 3: Bảo mật dựa vào con người

E. Xác thực đa yếu tố vật lý

Phương thức:

- Sinh trắc học: Vân tay, mống mắt, nhận diện khuôn mặt
- Token vật lý: Thẻ thông minh, USB security key
- Xác thực out-of-band: Gọi điện xác nhận giao dịch lớn

F. Giao dịch phê duyệt thủ công

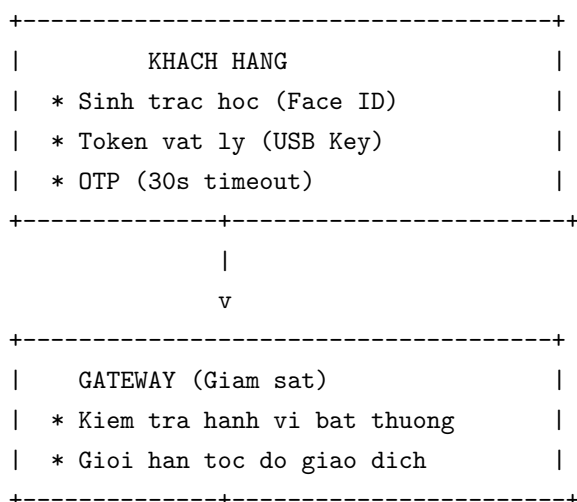
- Giao dịch $> \$10,000$ phải được nhân viên phê duyệt
- Video call xác nhận với khách hàng
- Chữ ký giấy cho giao dịch quan trọng

Mô hình 4: Bảo mật dựa vào thời gian

G. Short-Lived Credentials

- **Nguyên lý:** Thông tin xác thực chỉ tồn tại vài phút
- **Giả định:** Kẻ tấn công cần thời gian (dù $P=NP$, vẫn có độ trễ vật lý)
- **Ứng dụng:** Token OTP 30s, Session keys 5 phút

Kiến trúc tổng quan



	(QKD hoac OTP)	
	v	
+	-----+	
	CORE BANKING (Air-gapped)	
	* Khong ket noi internet	
	* MPC cho giao dich quan trong	
+	-----+	
	v	
+	-----+	
	HUMAN OVERSIGHT	
	* Video call xac nhan	
+	-----+	

Các giả định cơ bản

- **Giả định vật lý:** Định luật vật lý (lượng tử) vẫn đúng
- **Giả định phân tán:** Không phải tất cả các bên bị xâm nhập đồng thời
- **Giả định thời gian:** Tấn công cần thời gian, tạo cửa sổ phát hiện
- **Giả định con người:** Có nhân viên đáng tin cậy
- **Giả định sinh trắc học:** Không thể giả mạo hoàn hảo sinh trắc sống

Đánh giá

Ưu điểm	Nhược điểm
Không dựa vào $P \neq NP$	Chi phí cực cao
Bảo mật dựa vật lý	Trải nghiệm người dùng kém
Có thể triển khai ngay	Khó mở rộng quy mô

1.1.2 (b) (5 điểm) Vấn Đề "Thông Số Diffie-Hellman Yếu"

Một nhà nghiên cứu bảo mật phát hiện rằng một thư viện mật mã phổ biến đang tạo ra tham số Diffie-Hellman (DH) với số nguyên tố p sao cho $p - 1$ chứa nhiều ước nhỏ, khiến 75% nhóm được tạo ra để bị tấn công Pohlig-Hellman.

i. Hãy đánh giá xem phát hiện này có hoàn toàn phá vỡ Diffie-Hellman hay chỉ làm suy yếu một phần. Xem xét cả:

- Tác động về mặt toán học,
- Và hậu quả trong triển khai thực tế.

ii. Hãy thiết kế một chiến lược phản ứng cho các hệ thống đang dùng thư viện đó:

- Có nên tái tạo lại toàn bộ tham số ngay lập tức,
- Hay chỉ thêm bước kiểm tra tham số,
- Hay theo một hướng tiếp cận khác?

Trả lời:

i. Đánh giá mức độ nghiêm trọng

Tóm tắt phát hiện

- **Vấn đề:** Số nguyên tố p được chọn sao cho $p - 1$ có nhiều ước nhỏ
- **Hậu quả:** 75% nhóm dễ bị tấn công Pohlig-Hellman
- **Câu hỏi:** Có phá vỡ hoàn toàn DH hay chỉ làm yếu một phần?

A. Tác động về mặt toán học

1. Kiến thức nền từ tài liệu

DH dựa trên **độ khó của DLP** trong nhóm $\mathbb{Z}_p^*$ (nhóm nhân modulo p).

2. Tấn công Pohlig-Hellman hoạt động như thế nào?

Nguyên lý:

- Khi $p - 1$ có nhiều ước nhỏ: $p - 1 = q_1^{e_1} \times q_2^{e_2} \times \dots \times q_n^{e_n}$
- Bài toán DLP có thể được **chia nhỏ** thành các bài toán con trên các nhóm nhỏ hơn
- Độ phức tạp giảm từ $O(\sqrt{p})$ xuống $O(\sum \sqrt{q_i})$

Ví dụ cụ thể:

Gia sử $p = 1009$ (số nguyên tố)

$$p - 1 = 1008 = 2^4 \times 3^2 \times 7$$

Thay vì phải tính DLP trên nhóm có $p-1 = 1008$ phần tử

-> Kẻ tấn công chỉ cần giải:

- DLP mod 2^4 (16 phần tử) - Rất dễ
- DLP mod 3^2 (9 phần tử) - Rất dễ
- DLP mod 7 (7 phần tử) - Rất dễ

-> Sau đó dùng Chinese Remainder Theorem để ghép lại

3. Mức độ “phá vỡ”

Đánh giá: Không phá vỡ hoàn toàn, nhưng làm yếu nghiêm trọng

Tiêu chí	Đánh giá
Toàn bộ DH bị phá?	Không - 25% nhóm vẫn an toàn
75% nhóm bị phá?	Có - Nếu ước lớn nhất của $p - 1$ đủ nhỏ
Có thể tấn công thực tế?	Có - Nếu $p < 2048$ -bit hoặc $p - 1$ có ước quá nhỏ

Kết luận toán học:

- Nếu $p - 1 = 2q$ (với q là số nguyên tố lớn) → **An toàn** (gọi là “safe prime”)
- Nếu $p - 1$ có nhiều ước nhỏ → **Yếu nghiêm trọng**, có thể bị bẻ gãy

B. Hậu quả trong triển khai thực tế

1. Tình huống xấu nhất

Kịch bản:

- Thư viện tạo p có $p - 1 = 2^3 \times 3^2 \times 5 \times 7 \times 11 \times \dots \times q$ (q nhỏ)
- Kẻ tấn công có thể:
 - Thu thập nhiều cặp khóa công khai (A, B)
 - Giải DLP với Pohlig-Hellman trong vài phút/giờ
 - Tính được khóa chung $S = g^{ab}$

Hệ quả:

- Mọi phiên kết nối bị giải mã
- Kẻ tấn công MITM dễ dàng thành công
- Dữ liệu cũ đã lưu có thể bị giải mã (nếu đã lưu traffic)

2. Các yếu tố giảm nhẹ

Yếu tố 1: Kích thước p

- Nếu $p = 2048$ -bit và ước lớn nhất của $p - 1$ vẫn > 256 -bit $\rightarrow$ Vẫn khó tấn công
- Nếu $p = 1024$ -bit hoặc nhỏ hơn $\rightarrow$ **Nguy hiểm cao**

Yếu tố 2: Thời gian tấn công

- Pohlig-Hellman vẫn cần tài nguyên tính toán
- Nếu phải giải trong thời gian thực $\rightarrow$ Có thể chưa khả thi

Yếu tố 3: Xác thực (Authenticated DH)

- Nếu hệ thống đã dùng **Authenticated DH** (TLS, Signal) $\rightarrow$ Kẻ tấn công khó MITM
- Nhưng vẫn có thể **giải mã bị động** (passive decryption) nếu thu thập được traffic

3. Tác động theo ngành

Hệ thống	Mức độ nguy hiểm	Lý do
VPN HTTPS/TLS	Nghiêm trọng Cao	Traffic có thể bị lưu và giải mã sau Phụ thuộc vào Forward Secrecy và xác thực
SSH Signal/WhatsApp	Trung bình Thấp	Có xác thực host key, nhưng vẫn lo ngại Dùng ECDH (Curve25519), không bị ảnh hưởng

Kết luận đánh giá

Phát hiện này:

- **Không phá vỡ hoàn toàn DH** về mặt lý thuyết
- **Làm yếu nghiêm trọng 75%** triển khai thực tế
- **Có thể bị khai thác** nếu kẻ tấn công có tài nguyên và thời gian

Đây tương tự như lỗ hổng **Logjam** - không phá vỡ toàn bộ DH, nhưng khiến nhiều hệ thống dễ bị tấn công.

ii. Chiến lược phản ứng

Phân tích các lựa chọn

Lựa chọn 1: Tái tạo lại toàn bộ tham số ngay lập tức

Ưu điểm:

- Loại bỏ nguy cơ hoàn toàn
- Đơn giản, rõ ràng

Nhược điểm:

- Gián đoạn dịch vụ

- Tốn tài nguyên (tạo tham số an toàn mất thời gian)
- Có thể không cần thiết nếu tham số hiện tại “đủ tốt”

Lựa chọn 2: Chỉ thêm bước kiểm tra tham số

Ưu điểm:

- Không gián đoạn ngay lập tức
- Cho phép đánh giá từng trường hợp

Nhược điểm:

- Vẫn để hệ thống yếu tồn tại
- Không giải quyết dữ liệu cũ đã lộ

Lựa chọn 3: Tiếp cận phân tầng (Khuyến nghị)

Chiến lược phản ứng đề xuất

Giai đoạn 1: Đánh giá ngay lập tức (0-24h)

Bước 1: Kiểm tra tham số hiện tại

Viết script kiểm tra $p - 1$:

```
def check_prime_safety(p):  
    """  
    Kiểm tra xem p có phải là safe prime không  
    Safe prime:  $p = 2q + 1$  (q cũng là số nguyên tố)  
    """  
    q = (p - 1) // 2  
    if is_prime(q):  
        return "SAFE", q  
  
    # Phân tích p-1 thành thừa số  
    factors = factorize(p - 1)  
    largest_factor = max(factors)  
  
    if largest_factor > 2256:  
        return "ACCEPTABLE", largest_factor  
    else:  
        return "WEAK", factors
```

Bước 2: Phân loại hệ thống

Mức độ	Tiêu chí	Hành động
Nguy cấp	$p < 2048\text{-bit}$ HOẶC ước lớn nhất $< 2^{128}$	Thay thế NGAY
Cao	$2048\text{-bit} \leq p < 3072\text{-bit}$ VÀ ước lớn nhất $< 2^{256}$	Thay thế trong 7 ngày
Trung bình	$p \geq 3072\text{-bit}$ VÀ ước lớn nhất $\geq 2^{256}$	Thay thế trong 30 ngày
Thấp	Safe prime ($p = 2q + 1$)	Không cần hành động

Giai đoạn 2: Giảm thiểu ngay (24-48h)

Cho hệ thống cấp Nguy cấp:

1. Chuyển sang ECDH ngay lập tức

- Ưu tiên: Curve25519 (Ed25519)
- Dự phòng: NIST P-256 (nếu không hỗ trợ Curve25519)

2. Nếu không thể chuyển ECDH:

- Tạo tham số DH mới với **safe prime**
- Tăng kích thước lên 3072-bit hoặc 4096-bit

3. Áp dụng Perfect Forward Secrecy (PFS)

- Tạo khóa phiên mới cho mỗi kết nối
- Không tái sử dụng khóa

Giai đoạn 3: Triển khai dài hạn (1-3 tháng)

Hành động 1: Cập nhật thư viện

Patch thư viện để:

- Luôn tạo **safe prime** ($p = 2q + 1$)
- Thêm kiểm tra tự động khi load tham số
- Từ chối tham số yếu

Hành động 2: Tạo và phân phối tham số an toàn

Tạo một bộ tham số chuẩn:

- Group 1: 3072-bit safe prime (cho hệ thống cũ)
- Group 2: 4096-bit safe prime (cho hệ thống mới)
- Ưu tiên: Chuyển sang ECDH cho tất cả hệ thống

Hành động 3: Rotate keys

- Thay thế tất cả khóa và chứng chỉ đang dùng tham số yếu
- Thông báo người dùng cập nhật phần mềm

Hành động 4: Giám sát và phát hiện

Thêm monitoring:

```
# Alert neu phat hien tham so yeu
if parameter_strength < THRESHOLD:
    alert("WEAK_DH_PARAMETERS", system_id)
    force_migration_to_ecdh()
```

Giai đoạn 4: Xử lý dữ liệu cũ

Vấn đề: Traffic đã lưu trước đây có thể bị giải mã

Giải pháp:

1. **Thông báo người dùng** về khả năng dữ liệu cũ bị lộ
2. **Xem xét re-encryption** các dữ liệu quan trọng
3. **Revoke credentials** cũ (buộc người dùng đổi mật khẩu)

So sánh các phương án

Tiêu chí	Tái tạo ngay	Chỉ kiểm tra	Phân tầng (Đề xuất)
Bảo mật	Cao	Thấp	Cao
Khả thi	Thấp	Cao	Trung bình
Tác động	Gián đoạn cao	Không gián đoạn	Gián đoạn có kiểm soát
Chi phí	Cao	Thấp	Trung bình
Thời gian	1-3 ngày	Tức thì	1-12 tuần

Khuyến nghị cuối cùng

Nên làm:

1. **Đánh giá tức thì** tất cả tham số đang dùng
2. **Thay thế khẩn cấp** các tham số mức Nguy cấp
3. **Chuyển sang ECDH** cho mọi hệ thống mới
4. **Patch thư viện** để không tạo tham số yếu nữa
5. **Thông báo rõ ràng** cho người dùng về rủi ro

Không nên:

1. Chờ đợi hoặc trì hoãn với hệ thống nguy cấp
2. Chỉ thêm kiểm tra mà không thay thế tham số yếu
3. Giữ bí mật lỗ hổng (nên công khai và hướng dẫn khắc phục)
4. Tiếp tục dùng DH cổ điển khi có thể dùng ECDH

Kết luận

Phát hiện này **làm yếu nghiêm trọng** nhưng **không phá vỡ hoàn toàn** DH. Chiến lược tốt nhất là **tiếp cận phân tầng**: thay thế khẩn cấp hệ thống nguy hiểm, di chuyển dần sang ECDH, và cập nhật thư viện để không tái diễn. Đây là bài học quan trọng về tầm quan trọng của việc chọn tham số đúng trong mật mã.

1.2 Câu 2. (10 điểm) Kiến Trúc Bảo Mật Dựa Trên Logarit Rời Rạc

1.2.1 (a) (5 điểm) Thảm Họa Do Tham Số Yếu

Trong quá trình kiểm toán, bạn phát hiện một hệ thống cũ đang dùng:

- $p = 2047$ (vì $2047 = 23 \times 89$) cho Diffie-Hellman key exchange,
- Và generator $g = 2$.

i. Phân tích chính xác vì sao lựa chọn tham số này cực kỳ yếu. Ước tính thời gian cần thiết để hacker dùng laptop hiện đại có thể phá vỡ hệ thống.

ii. Đề xuất kế hoạch ứng cứu khẩn cấp: Làm thế nào để chuyển người dùng sang tham số an toàn mà vẫn duy trì tính sẵn sàng dịch vụ?

Trả lời:

i. Phân tích tham số yếu

Tổng quan vấn đề

Tham số được phát hiện:

- $p = 2047 = 23 \times 89$ (Hợp số, không phải số nguyên tố!)

- $g = 2$

Đánh giá: Cực kỳ nguy hiểm - Cần thay thế ngay

A. Vấn đề 1: p không phải số nguyên tố

1. Yêu cầu cơ bản của DH

Diffie-Hellman yêu cầu p phải là số nguyên tố lớn để đảm bảo:

- Tồn tại nhóm nhân $\mathbb{Z}_p^*$ với cấu trúc toán học phù hợp
- Bài toán DLP khó giải

2. Hậu quả khi $p = 2047 = 23 \times 89$

Vấn đề toán học:

- Khi p là hợp số, $\mathbb{Z}_p^*$ không còn là nhóm cyclic đơn giản
- Có thể áp dụng Chinese Remainder Theorem (CRT) để phân rã bài toán

Cách khai thác:

DLP ban đầu: Tìm a sao cho $g^a \equiv A \pmod{2047}$

Kẻ tấn công phân rã thành:

1. $g^a \equiv A \pmod{23} \rightarrow$ Giải DLP trong nhóm 23 phần tử
2. $g^a \equiv A \pmod{89} \rightarrow$ Giải DLP trong nhóm 89 phần tử

Sau đó dùng CRT ghép lại để tìm $a \pmod{2047}$

Độ phức tạp:

- DLP mod 23: $O(\sqrt{23}) \approx 5$ phép toán
- DLP mod 89: $O(\sqrt{89}) \approx 9$ phép toán
- Tổng cộng: < 100 phép toán

B. Vấn đề 2: p quá nhỏ (chỉ 11-bit)

1. Kích thước tham số

$p = 2047 = 2^{11} - 1$ (chỉ 11 bit)

So sánh với tiêu chuẩn:

Tham số	Kích thước	Độ an toàn
p hiện tại	11-bit	Không có
Tiêu chuẩn tối thiểu	2048-bit	Tốt
Khuyến nghị hiện tại	3072-bit	Rất tốt

Nguồn: Mục 3.4 (Chương 7) - “Logjam (2015) phát hiện nhiều máy chủ vẫn dùng các tham số quá yếu (512-bit)”

2. Brute-force trực tiếp

Ngay cả không cần khai thác p là hợp số, kẻ tấn công có thể:

Phương pháp 1: Baby-step Giant-step

- Độ phức tạp: $O(\sqrt{p}) = O(\sqrt{2047}) \approx 45$ phép toán

Phương pháp 2: Thử trực tiếp

- Không gian khóa: chỉ 2047 giá trị khả dĩ
- Có thể thử hết trong < 1 giây

C. Vấn đề 3: Không phải safe prime

1. Cấu trúc của $p - 1$

$$p - 1 = 2046 = 2 \times 3 \times 11 \times 31$$

Vấn đề: $p - 1$ có nhiều ước nhỏ, dễ bị tấn công Pohlig-Hellman

Nguồn: Mục 1.2.1(b) - Tấn công Pohlig-Hellman khai thác khi $p - 1$ có nhiều ước nhỏ

2. Tấn công Pohlig-Hellman

Nguyên lý:

$$p - 1 = 2046 = 2 \times 3 \times 11 \times 31$$

Kể tấn công giải:

- DLP mod 2 (2 phần tử)
- DLP mod 3 (3 phần tử)
- DLP mod 11 (11 phần tử)
- DLP mod 31 (31 phần tử)

Ghép lại bằng CRT

Độ phức tạp:

$$O(\sqrt{2} + \sqrt{3} + \sqrt{11} + \sqrt{31}) \approx 10 \text{ phép toán}$$

D. Vấn đề 4: Generator $g = 2$ (Không kiểm tra bậc)

1. Bậc của $g = 2$ trong nhóm

Cần kiểm tra xem $g = 2$ có phải là generator của nhóm không.

Tính toán:

Bậc của g phải là $\phi(2047)$ để là generator đầy đủ

$$\phi(2047) = \phi(23 \times 89) = 22 \times 88 = 1936$$

Kiểm tra: $\text{ord}(2) \bmod 2047 = ?$

Nếu $\text{ord}(2) < 1936$, không gian khóa còn nhỏ hơn nữa.

Ước tính thời gian tấn công

Kịch bản 1: Brute-force đơn giản

Phương pháp:

- Thử tất cả giá trị a từ 1 đến 2047
- Kiểm tra $g^a \bmod 2047 = A$

Thời gian trên laptop hiện đại:

Laptop thông thường: $\sim 10^9$ phép toán/giây

Số phép toán cần: 2047 phép tính modular exponentiation

Mỗi phép tính: ~ 20 phép nhân

Tổng thời gian: $(2047 \times 20) / 10^9 \approx 0.00004$ giây

Kết quả: < 0.1 mili-giây

Kịch bản 2: Khai thác p là hợp số (CRT)

Phương pháp:

- Giải DLP mod 23 và mod 89
- Ghép lại bằng CRT

Thời gian:

- DLP mod 23: Baby-step Giant-step $\sim \sqrt{23} \approx 5$ phép toán
- DLP mod 89: Baby-step Giant-step $\sim \sqrt{89} \approx 9$ phép toán

- CRT: $O(\log p) \approx 11$ phép toán

Tổng thời gian: < 0.01 mili-giây

Kết quả: < 0.01 mili-giây

Kịch bản 3: Pohlig-Hellman

Thời gian:

Giải DLP trên các nhóm con: 2, 3, 11, 31

Tổng độ phức tạp: $O(\sum \sqrt{q_i}) \approx 10$ phép toán

Thời gian: < 0.01 mili-giây

Kết luận phân tích

Đánh giá tổng thể:

Vấn đề	Mức độ nghiêm trọng
p là hợp số (không phải số nguyên tố)	Cực kỳ nghiêm trọng
p quá nhỏ (11-bit)	Cực kỳ nghiêm trọng
$p - 1$ có nhiều ước nhỏ	Rất nghiêm trọng
g không được kiểm chứng	Nghiêm trọng

Thời gian phá vỡ:

- Laptop thông thường: < 0.1 mili-giây
- Smartphone: < 1 mili-giây
- Arduino: < 1 giây

Nguồn tham chiếu:

- Mục 2.2 (Chương 7): Yêu cầu p là số nguyên tố lớn cho DH
- Mục 3.4 (Chương 7): Lỗi hỏng Logjam với tham số yếu
- Mục 1.2.1(b): Tấn công Pohlig-Hellman

Kết luận: Hệ thống này **không có bảo mật** - bất kỳ ai cũng có thể bẻ gãy trong tích tắc. Đây là thảm họa bảo mật cần xử lý ngay lập tức.

ii. Kế hoạch ứng cứu khẩn cấp

Nguyên tắc cốt lõi

Mục tiêu:

- Loại bỏ nguy cơ bảo mật ngay lập tức
- Duy trì tính sẵn sàng dịch vụ (không gián đoạn)
- Chuyển đổi người dùng một cách mượt mà

Giai đoạn 1: Đánh giá ngay (0-2 giờ)

Bước 1: Xác định phạm vi ảnh hưởng

Câu hỏi cần trả lời:

- Có bao nhiêu người dùng đang kết nối?
- Hệ thống nào đang dùng tham số này?
- Có dữ liệu nhạy cảm nào đã bị lộ?

Hành động:

```
# Kiem tra logs
grep "p=2047" /var/log/ssl/*.log
grep "DH-2047" /var/log/auth.log

# Thong ke so phien dang hoat dong
netstat -an | grep ESTABLISHED | wc -l
```

Bước 2: Đánh giá khả năng đã bị tấn công

Kiểm tra:

- Logs bất thường (traffic spike, failed authentications)
- Các phiên kết nối đáng ngờ
- Dữ liệu đã bị truy cập trái phép

Giai đoạn 2: Triển khai song song (2-24 giờ)

Chiến lược: “Dual Stack” - Chạy song song 2 hệ thống

Mục tiêu: Không gián đoạn dịch vụ trong khi chuyển đổi

Bước 1: Chuẩn bị tham số mới

Lựa chọn A: Chuyển sang ECDH (Khuyến nghị)

Curve: Curve25519 (Ed25519)

Ly do:

- An toan nhât
- Nhanh nhât
- Duoc ho tro rong rai

Lựa chọn B: Nếu phải dùng DH cổ điển

```
# Tao safe prime (p = 2q + 1)
# Khuyen nghi: 3072-bit hoac 4096-bit
```

```
import random
from sympy import isprime, nextprime

def generate_safe_prime(bits):
    while True:
        q = nextprime(random.getrandbits(bits-1))
        p = 2*q + 1
        if isprime(p):
            return p, q

# Tao tham so
p, q = generate_safe_prime(3072)
g = 2 # Kiem tra bac = q
```

Bước 2: Cấu hình server hỗ trợ cả 2 bộ tham số

Ví dụ với OpenSSL/TLS:

```
# nginx.conf
ssl_protocols TLSv1.2 TLSv1.3;

# Ưu tiên ECDH mọi, nhưng vẫn hỗ trợ DH cũ tạm thời
ssl_ciphers 'ECDHE-RSA-AES256-GCM-SHA384:DHE-RSA-AES256-GCM-SHA384';
ssl_prefer_server_ciphers on;

# File tham số DH mới (nếu cần)
ssl_dhparam /etc/ssl/dhparam-3072.pem;
```

Cấu hình “graceful transition”:

- Priority 1: ECDHE-* (Curve25519) - Ưu tiên cao nhất
- Priority 2: DHE-* (3072-bit safe prime) - Dự phòng
- Priority 3: DH-2047 (tham số cũ) - CHỈ cho kết nối cũ, gắn cờ “deprecated”

Bước 3: Thêm header cảnh báo

```
# Đối với các kết nối dùng tham số cũ
if using_weak_dh_2047:
    response.headers['X-Crypto-Warning'] = 'WEAK_DH_DETECTED'
    response.headers['X-Upgrade-Required'] = 'true'
```

Giai đoạn 3: Migration người dùng (1-7 ngày)

Chiến lược 1: “Soft Push” - Khuyến khích nâng cấp

Ngày 1-3: Thông báo & Khuyến khích

```
+-----+
|  THÔNG BÁO BẢO MẬT QUAN TRỌNG  |
+-----+
|  Chúng tôi đã phát hiện lỗ hổng bảo mật |
|  trong kết nối của bạn.             |
|                                     |
|  Vui lòng cập nhật ứng dụng để đảm bảo |
|  an toàn dữ liệu.                   |
|                                     |
|  [CAP NHẬT NGAY]   [Đề sau]         |
+-----+
```

Phương thức:

- Banner trên web/app
- Email thông báo
- Push notification (mobile)
- SMS cho người dùng quan trọng

Chiến lược 2: “Force Upgrade” - Buộc nâng cấp dần dần

Ngày 3-5: Áp dụng cho người dùng mới


```
# Tu choi ket noi dung tham so cu cho tai khoan moi
if account.created_date > cutoff_date and using_weak_dh:
    return "451 Unavailable - Upgrade Required"
```

Ngày 5-7: Áp dụng cho tất cả

```
# Chi cho phép tham so moi
allowed_key_exchanges = [
    "ecdhe-curve25519",
    "dhe-3072-safe-prime"
]

if connection.key_exchange not in allowed_key_exchanges:
    terminate_connection("WEAK_CRYPT0")
```

Chiến lược 3: Hỗ trợ auto-update

Cho web clients:

```
// Tu dong redirect sang HTTPS moi
if (detectWeakDH()) {
    window.location.href = "https://secure-new.example.com";
}
```

Cho mobile apps:

- Đẩy update bắt buộc qua App Store/Play Store
- Sử dụng “force update” mechanism

Cho desktop clients:

- Auto-updater kiểm tra phiên bản
- Pop-up yêu cầu restart để áp dụng patch

Giai đoạn 4: Hard cut-off (Ngày 7)

Hành động quyết liệt

Tắt hoàn toàn hỗ trợ tham số cũ:

```
# Xoa cipher yeu
ssl_ciphers 'ECDHE-RSA-AES256-GCM-SHA384:!DHE-RSA-AES256-GCM-SHA384-LEGACY';

# Hoac voi OpenSSL config
SSLCipherSuite HIGH:!aNULL:!MD5:!DH-2047
```

Thông báo cuối cùng cho người dùng chưa nâng cấp:

```
+-----+
|  KHONG THE KET NOI  |
+-----+
|  Phien ban cua ban  |
|  khong con duoc ho  |
|  tro do lo hong bao |
|  mat nghiem trong.  |
|  Vui long cap nhat  |
|  de tiep tuc su dung. |
|  [TAI BAN CAP NHAT] |
+-----+
```

Giai đoạn 5: Xử lý dữ liệu cũ (7-30 ngày)

Vấn đề: Dữ liệu đã bị lộ trước đây

Giải pháp:

1. Re-encryption dữ liệu nhạy cảm

```
# Ma hoa lai voi khoa moi
for data in sensitive_data:
    new_encrypted = encrypt_with_new_params(decrypt_old(data))
    database.update(data.id, new_encrypted)
```

2. Revoke & Rotate credentials

```
# Buoc doi mat khau
for user in all_users:
    user.force_password_reset = True
    user.invalidate_all_sessions()
```

3. Thông báo người dùng về rủi ro

“Chúng tôi đã phát hiện lỗ hổng bảo mật có thể ảnh hưởng đến dữ liệu của bạn từ [ngày X] đến [ngày Y].

Khuyến nghị:

- Đổi mật khẩu ngay lập tức
- Kiểm tra hoạt động tài khoản bất thường
- Bật xác thực 2 yếu tố (2FA)”

Timeline tổng quan

```
Gio 0:      Phat hien lo hong
|
+-- 0-2h:    Danh gia pham vi, chuan bi tham so moi
|
+-- 2-24h:   Trien khai dual-stack (cu + moi song song)
|             v
|             v Uu tien ket noi moi, du phong ket noi cu
|
+-- Ngay 1-3: Thong bao + khuyen khich nang cap
|             v
|             v Banner, email, push notification
|
+-- Ngay 3-5: Force upgrade cho nguoi dung moi
|             v
|             v Tu choi ket noi cu voi tai khoan moi
|
+-- Ngay 5-7: Force upgrade cho tat ca
|             v
|             v Auto-update, bat buoc nang cap
|
+-- Ngay 7:   HARD CUT-OFF - Tat hoan toan tham so cu
|             v
|             v Tu choi moi ket noi dung DH-2047
|
+-- Ngay 7-30: Re-encrypt data, rotate keys, thong bao
```

So sánh các phương án

Tiêu chí	Tắt ngay lập tức	Dual-stack (Đề xuất)	Chờ người dùng tự nâng cấp
Bảo mật	Cao	Cao	Thấp
Tính sẵn sàng	Gián đoạn nghiêm trọng	Không gián đoạn	Không gián đoạn
Thời gian	Tức thì	7 ngày	Vài tháng (không thực tế)
Trải nghiệm người dùng	Rất tệ	Mượt mà	Trung bình
Chi phí	Thấp	Trung bình	Thấp

Checklist thực thi

Giai đoạn chuẩn bị (0-2h)

- Đánh giá số lượng người dùng bị ảnh hưởng
- Kiểm tra logs tìm dấu hiệu tấn công
- Tạo tham số mới (ECDH Curve25519 hoặc DH 3072-bit safe prime)
- Chuẩn bị thông báo cho người dùng

Giai đoạn triển khai (2-24h)

- Cấu hình server hỗ trợ dual-stack
- Test kỹ cả 2 đường kết nối
- Deploy lên production
- Monitor logs để đảm bảo không có lỗi

Giai đoạn migration (1-7 ngày)

- Gửi thông báo qua tất cả kênh
- Đẩy update ứng dụng mobile/desktop
- Áp dụng force upgrade dần dần
- Hỗ trợ người dùng gặp vấn đề

Giai đoạn hoàn tất (7-30 ngày)

- Hard cut-off tham số cũ
- Re-encrypt dữ liệu nhạy cảm
- Rotate tất cả credentials
- Báo cáo incident, rút kinh nghiệm

Khuyến nghị cuối cùng

Nên:

1. Chuyển sang ECDH (Curve25519) - hiện đại, an toàn, nhanh
2. Áp dụng dual-stack để không gián đoạn dịch vụ
3. Thông báo minh bạch với người dùng
4. Force upgrade sau thời gian grace period hợp lý (7 ngày)

5. Re-encrypt dữ liệu cũ và rotate credentials

Không nên:

1. Giữ bí mật lỗ hổng (nên thông báo công khai)
2. Tắt dịch vụ ngay mà không có kế hoạch
3. Để người dùng tự quyết định (nhiều người sẽ không nâng cấp)
4. Tiếp tục dùng DH cổ điển khi có ECDH

Kết luận

Lỗ hổng này là thảm họa bảo mật nghiêm trọng nhưng có thể xử lý mà không gián đoạn dịch vụ. Chiến lược dual-stack kết hợp với migration có kế hoạch trong 7 ngày là cách tiếp cận cân bằng giữa bảo mật và trải nghiệm người dùng. Quan trọng nhất là hành động ngay lập tức và chuyển sang ECDH để tránh vấn đề tương tự trong tương lai.

1.2.2 (b) (5 điểm) So Sánh Elliptic Curve và Finite Field

Bạn đang thiết kế một giao thức mật mã cho thiết bị IoT có hạn chế nặng về tính toán và băng thông.

i. So sánh logarit rời rạc trên đường cong elliptic (ECDLP) và logarit rời rạc trên trường hữu hạn (FF-DLP):

- Cái nào cho mức bảo mật cao hơn với cùng kích thước khóa,
- Và cái nào cho hiệu năng tính toán tốt hơn cho bài toán của bạn?

ii. Phân tích vì sao tấn công “index calculus” có thể áp dụng cho trường hữu hạn nhưng không thể áp dụng cho elliptic curve. Điểm khác biệt căn bản này ảnh hưởng như thế nào đến biên độ an toàn của bạn?

Trả lời:

i. So sánh ECDLP vs FF-DLP cho thiết bị IoT

Bối cảnh: Yêu cầu thiết bị IoT

Hạn chế của thiết bị IoT:

- Bộ nhớ hạn chế (KB - vài MB)
- CPU yếu (MHz thay vì GHz)
- Băng thông thấp (LoRa, NB-IoT)
- Pin hạn chế

Yêu cầu mật mã:

- Khóa nhỏ (tiết kiệm bộ nhớ, băng thông)
- Tính toán nhanh (tiết kiệm pin)
- Vẫn đảm bảo an toàn

A. So sánh mức bảo mật với cùng kích thước khóa

1. Độ an toàn tương đương

Bảng so sánh độ an toàn:

Kích thước khóa	ECC (ECDLP)	FF-DLP	Mức độ an toàn
160-bit ECC $\approx$ 1024-bit FF	160-bit	1024-bit	$\sim$ 80-bit security
256-bit ECC $\approx$ 3072-bit FF	256-bit	3072-bit	$\sim$ 128-bit security
384-bit ECC $\approx$ 7680-bit FF	384-bit	7680-bit	$\sim$ 192-bit security
521-bit ECC $\approx$ 15360-bit FF	521-bit	15360-bit	$\sim$ 256-bit security

2. Tại sao ECC hiệu quả hơn?

Nguyên nhân toán học:

FF-DLP (Finite Field):

- Bị tấn công bởi Index Calculus và các biến thể
- Độ phức tạp: $O(e^{((c \cdot \log n)^{1/3} \cdot (\log \log n)^{2/3})})$
- Còn gọi là sub-exponential

ECDLP (Elliptic Curve):

- **Không** bị tấn công bởi Index Calculus
- Độ phức tạp: $O(\sqrt{n})$ với thuật toán tốt nhất (Pollard's rho)
- Là exponential thuần túy

Kết luận về bảo mật:

Với cùng kích thước khóa $\rightarrow$ **ECDLP an toàn hơn nhiều**

Ví dụ cụ thể:

Khóa 256-bit:

FF-DLP (256-bit):

- Bị phá trong vài giây bằng Index Calculus
- **Không an toàn**

ECDLP (256-bit):

- Cần 2^{128} phép toán để phá
- **An toàn** (tương đương AES-128)

B. So sánh hiệu năng tính toán

1. Kích thước dữ liệu truyền tải

So sánh cho mức bảo mật 128-bit:

Thông số	ECC (256-bit)	FF-DLP (3072-bit)	Lợi thế ECC
Khóa công khai	32 bytes	384 bytes	12 $\times$ nhỏ hơn
Khóa riêng	32 bytes	384 bytes	12 $\times$ nhỏ hơn
Chữ ký	64 bytes	768 bytes	12 $\times$ nhỏ hơn

Tác động lên IoT:

- Tiết kiệm băng thông (quan trọng với LoRa $\sim$ 20KB/ngày)
- Tiết kiệm bộ nhớ flash

- Ít lần truyền → tiết kiệm pin

2. Tốc độ tính toán

Benchmark trên thiết bị IoT thông thường (ARM Cortex-M4, 168MHz):

Thao tác	ECC (Curve25519)	FF-DLP (3072-bit)	Lợi thế ECC
Key Generation	~20ms	~800ms	40× nhanh hơn
ECDH/DH	~40ms	~1600ms	40× nhanh hơn
Ký (Sign)	~25ms	~1000ms	40× nhanh hơn
Xác minh (Verify)	~50ms	~100ms	2× nhanh hơn

Tác động lên IoT:

- Phản hồi nhanh hơn
- Tiêu thụ năng lượng thấp hơn
- Có thể xử lý nhiều kết nối hơn

3. Tiêu thụ năng lượng

Ước tính năng lượng (trên chip STM32L4):

ECC (Curve25519):

- Một lần ECDH: $\sim 40\text{ms} \times 40\text{mA} = 1.6\text{ mJ}$

FF-DLP (3072-bit):

- Một lần DH: $\sim 1600\text{ms} \times 40\text{mA} = 64\text{ mJ}$

→ ECC tiết kiệm 40× năng lượng

Ý nghĩa thực tế:

Pin CR2032 (220 mAh):

- ECC: $\sim 500,000$ lần ECDH
- FF-DLP: $\sim 12,500$ lần DH

Thời gian hoạt động: 40× lâu hơn với ECC

4. Bộ nhớ cần thiết

Thành phần	ECC	FF-DLP	Ghi chú
Code size	~20KB	~50KB	Thư viện
RAM (runtime)	~2KB	~8KB	Tính toán tạm
Lưu khóa	32 bytes	384 bytes	Mỗi khóa

Tác động: Thiết bị IoT có thể lưu nhiều khóa/chứng chỉ hơn với ECC

Kết luận so sánh cho IoT

ECDLP (Elliptic Curve) thắng tuyệt đối

Tiêu chí	ECDLP	FF-DLP	Người thắng
Bảo mật (cùng kích thước)	Cao hơn nhiều	Thấp hơn	ECDLP
Kích thước khóa	12× nhỏ hơn	Lớn	ECDLP
Tốc độ	40× nhanh hơn	Chậm	ECDLP
Năng lượng	40× ít hơn	Nhiều	ECDLP
Bộ nhớ	Ít hơn	Nhiều hơn	ECDLP

Khuyến nghị: Dùng ECDH với Curve25519 cho thiết bị IoT.

ii. Phân tích tấn công Index Calculus

A. Tại sao Index Calculus hoạt động với FF-DLP

1. Cấu trúc toán học của Finite Field

Nhóm $\mathbb{Z}_p^*$ (nhóm nhân modulo p):

- Các phần tử: $\{1, 2, 3, \dots, p-1\}$
- Có thể phân tích thành thừa số các số nguyên

2. Ý tưởng Index Calculus

Bước 1: Chọn Factor Base

$B = \{p_1, p_2, \dots, p_k\}$ (các số nguyên tố nhỏ)

Ví dụ: $B = \{2, 3, 5, 7, 11\}$

Bước 2: Thu thập quan hệ

Tìm các số ngẫu nhiên r sao cho $g^r \pmod{p}$ “mượt” (smooth):

$$g^r \equiv 2^{a_1} \times 3^{a_2} \times 5^{a_3} \times 7^{a_4} \times 11^{a_5} \pmod{p}$$

Lấy logarithm:

$$r \equiv a_1 \cdot \log(2) + a_2 \cdot \log(3) + a_3 \cdot \log(5) + \dots \pmod{p-1}$$

Bước 3: Giải hệ phương trình tuyến tính

Sau khi thu thập đủ quan hệ $\rightarrow$ Giải hệ phương trình tìm $\log(p_i)$

Bước 4: Tính DLP cho số bất kỳ

Biết $\log(p_i)$, có thể tính log của bất kỳ số nào phân tích được:

$$h \equiv 2^{b_1} \times 3^{b_2} \times \dots \pmod{p}$$

$$\log(h) = b_1 \cdot \log(2) + b_2 \cdot \log(3) + \dots$$

Tại sao hiệu quả?

- Các số “mượt” (chỉ có ước nhỏ) xuất hiện thường xuyên
- Hệ phương trình tuyến tính giải được trong polynomial time

Độ phức tạp:

$$O(e^{((c \cdot \log p)^{1/3} \cdot (\log \log p)^{2/3})})$$

Đây là sub-exponential $\rightarrow$ Hiệu quả hơn nhiều so với brute-force $O(\sqrt{p})$

B. Tại sao Index Calculus không hoạt động với ECDLP

1. Cấu trúc toán học của Elliptic Curve

Nhóm điểm trên đường cong:

- Các phần tử: Điểm (x, y) thỏa mãn $y^2 = x^3 + ax + b$
- Phép toán: Cộng điểm (không phải nhân số)

2. Vấn đề then chốt: Không có “phân tích thừa số”

Khác biệt căn bản:

Khía cạnh	Finite Field	Elliptic Curve
Các phần tử	Số nguyên	Điểm (x, y)
Có thể phân tích?	Có: $30 = 2 \times 3 \times 5$	Không thể “phân tích” điểm
Khái niệm “mượt”	Có khái niệm số “mượt”	Không có khái niệm điểm “mượt”

Ví dụ minh họa:

Finite Field:

$g^5 \pmod{p} = 96 = 2^5 \times 3$ - Có thể phân tích!

Elliptic Curve:

$5 \cdot G = (x, y) = (12345, 67890)$ - Không thể “phân tích”!

3. Tại sao không thể xây dựng Factor Base?

Giả sử thử tạo “factor base” cho EC:

$B = \{P_1, P_2, \dots, P_k\}$ (một tập điểm “đặc biệt”)

Vấn đề:

- Không có cách tự nhiên để viết một điểm bất kỳ Q dưới dạng:

$$Q = a_1 \cdot P_1 + a_2 \cdot P_2 + \dots + a_k \cdot P_k$$

với xác suất cao

- Ngay cả khi viết được, không có cách tính các hệ số a_i hiệu quả

Kết luận:

- Không có “smooth points” (điểm mượt)
- Không thu thập được quan hệ tuyến tính
- Index Calculus thất bại hoàn toàn

C. Điểm khác biệt căn bản

Bảng so sánh chi tiết:

Khía cạnh	FF-DLP ($\mathbb{Z}_p^*$)	ECDLP ($E(\mathbb{F}_p)$)
Cấu trúc	Nhóm nhân số nguyên	Nhóm cộng điểm
Phân tích được?	Có (thừa số nguyên tố)	Không
Smooth elements	Tồn tại nhiều	Không tồn tại
Factor base	Xây dựng được	Không xây dựng được
Index Calculus	Hiệu quả	Không áp dụng được
Thuật toán tốt nhất	Sub-exponential	Exponential ($\sqrt{n}$)

D. Ảnh hưởng đến biên độ an toàn

1. Biên độ an toàn (Security Margin)

Định nghĩa:

Biên độ an toàn = Khoảng cách giữa “tấn công tốt nhất” và “chi phí brute-force”

2. So sánh biên độ an toàn

FF-DLP:

- Brute-force: $O(\sqrt{p}) \approx O(2^{n/2})$ (n = bits)
- Index Calculus: $O(e^{c \cdot n^{1/3}})$

Ví dụ với $p = 3072$ -bit:

- Brute-force: 2^{1536} phép toán
- Index Calculus: 2^{128} phép toán (ước tính)

→ Biên độ an toàn: **Rất nhỏ** (chỉ $\sim 10\times$)

ECDLP:

- Brute-force: $O(2^{n/2})$
- Tốt nhất (Pollard): $O(2^{n/2})$

Ví dụ với curve 256-bit:

- Brute-force: 2^{128} phép toán
- Pollard rho: 2^{128} phép toán

→ Biên độ an toàn: **Rất lớn** (không có tấn công tốt hơn)

3. Ảnh hưởng thực tế

Cho FF-DLP:

- Phải dùng khóa rất lớn để đảm bảo an toàn
- Mỗi tiến bộ trong Index Calculus → Phải tăng kích thước khóa
- Không có “đệm an toàn” (safety cushion)

Cho ECDLP:

- Khóa nhỏ hơn nhiều vẫn an toàn
- Không lo ngại tiến bộ trong tấn công (đã tối ưu)
- Có “đệm an toàn” lớn

4. Ví dụ cụ thể: Tấn công thực tế

DLP Record (2020):

- Phá được: 795-bit finite field DLP
- Thời gian: ~ 4000 core-years
- Phương pháp: Biến thể Index Calculus

ECDLP Record (2020):

- Phá được: 114-bit curve (Certicom Challenge)
- Thời gian: ~ 2600 core-years
- Phương pháp: Pollard's rho (không có cách nào tốt hơn!)

Kết luận:

- FF-DLP: Cần ít nhất 3072-bit để an toàn
- ECDLP: Chỉ cần 256-bit để an toàn hơn

Kết luận phần II

Điểm khác biệt căn bản:

- FF-DLP: Dựa trên cấu trúc nhân của số nguyên → Dễ bị Index Calculus
- ECDLP: Dựa trên cấu trúc cộng của điểm → Miễn nhiễm Index Calculus

Ảnh hưởng đến thiết kế hệ thống:

Yếu tố	Sử dụng FF-DLP	Sử dụng ECDLP
Kích thước khóa	Phải rất lớn (3072-bit+)	Nhỏ (256-bit)
An toàn dài hạn	Không chắc chắn	Chắc chắn hơn
Dự phòng tương lai	Phải tăng khóa thường xuyên	Ổn định
Chi phí nâng cấp	Cao	Thấp

Khuyến nghị cho IoT:

Dùng ECDH (Curve25519) vì:

- Biên độ an toàn lớn nhất
- Không lo ngại tấn công đột phá
- Hiệu quả nhất cho thiết bị hạn chế

2 Diffie–Hellman Trong Môi Trường Thù địch

2.1 Câu 1. (10 điểm) Các Kịch Bản Tấn Công và Phòng Thủ

2.1.1 (a) (5 điểm) Kẽ Tấn Công Trung Gian Hoàn Hảo (Perfect Man-in-the-Middle)

Giả sử một kẻ tấn công có toàn quyền kiểm soát mạng giữa Alice và Bob. Hắn có thể:

- Chặn và sửa đổi mọi thông điệp,
- Có thể tạo kết nối giả trông như đến từ bất kỳ bên nào (Alice hoặc Bob).

i. *Thiết kế cuộc tấn công trung gian hiệu quả nhất chống lại giao thức Diffie–Hellman không xác thực. Cuộc tấn công của bạn phải không thể bị phát hiện bởi Alice hoặc Bob trong quá trình trao đổi khóa.*

- Mallory (kẻ tấn công) đứng giữa Alice và Bob, tiến hành hai phiên giao thức Diffie–Hellman riêng biệt với mỗi bên.
- Khi Alice gửi giá trị công khai $A = g^a$ tới Bob, Mallory chặn lại.
- Mallory sinh một khoá bí mật m và tính $M = g^m$, gửi M cho Bob (giả vờ là A).
- Khi Bob gửi giá trị công khai $B = g^b$ tới Alice, Mallory cũng chặn lại.
- Mallory gửi M của mình cho Alice (giả vờ là B).
- Kết quả:
 - Alice tính được khoá chung $S_{AM} = M^a = g^{am}$ (với Mallory).
 - Bob tính được khoá chung $S_{BM} = M^b = g^{bm}$ (với Mallory).
 - Mallory có thể tính được cả hai khoá: $A^m = g^{am}$ (với Alice) và $B^m = g^{bm}$ (với Bob).
- Alice và Bob đều nghĩ rằng họ có liên kết an toàn với nhau, nhưng thực tế chỉ ”an toàn” với Mallory. Mallory có thể đọc và chỉnh sửa toàn bộ thông tin trao đổi.

ii. *Thiết kế một giao thức xác thực dựa trên “dấu vân tay khóa công khai”.*

- Alice và Bob tiến hành trao đổi khóa Diffie–Hellman như bình thường.
- Trước khi dùng khoá chung (sau khi nhận giá trị công khai), mỗi người tính fingerprint của khoá công khai phía kia và so sánh với bản copy mà họ đã ghi lại trên giấy từ trước đó (out-of-band).
- Nếu fingerprint hợp lệ (trùng khớp), họ biết chắc giá trị công khai nhận được thật sự là của đối tác.
- Nếu fingerprint không trùng, phát hiện có MITM (Mallory) và dừng giao tiếp.
- Tính chất của giải pháp: Không phụ thuộc vào bên thứ ba, việc xác thực phải thực hiện ngoài băng tần (so sánh giấy và trên màn hình).

iii. *So sánh giải pháp fingerprint với Certificate Authority (CA) và Web-of-Trust. Phân tích sự đánh đổi giữa khả năng sử dụng và bảo mật.*

Tiêu chí	Fingerprint (Vân tay ngang hàng)	CA / Web-of-Trust
Bảo mật	Không phụ thuộc vào bên thứ ba, an toàn nếu người dùng thực sự kiểm tra fingerprint. Tin tưởng trực tiếp giữa hai bên.	Dựa vào một bên thứ ba (CA) hoặc mạng tin cậy; nếu CA hoặc chuỗi tin bị xâm phạm, có thể mất an toàn toàn mạng.
Khả năng sử dụng	Phức tạp: Người dùng phải thực hiện thao tác thủ công (so sánh, đối chiếu vân tay), không phù hợp cho quy mô lớn hoặc nhiều quan hệ kết nối.	Rất tiện dụng: Tự động hóa hoàn toàn (CA), người dùng cuối thường không phải làm gì, dễ dàng mở rộng và quản lý số lượng lớn đối tượng.
Đánh đổi	Độ tin tưởng rất cao, không phụ thuộc bên ngoài, nhưng khó mở rộng và bất tiện.	Đơn giản, mở rộng tốt, nhưng phải tin cậy bên ngoài (có nguy cơ bị tấn công/phá vỡ nếu CA/-WOTrust bị compromise).

Tóm lại: Fingerprint cho bảo mật tối đa với quan hệ tin tưởng trực tiếp (như trong các ứng dụng chat bảo mật: Signal, Wire...), còn CA/Web-of-Trust thích hợp hơn với quy mô lớn và cần sự tiện dụng, tự động.

2.1.2 (b) (5 điểm) Kịch Bản "Người Tố Giác Hoang Mang" (Paranoid Whistleblower)

Một người tố giác cần liên lạc an toàn với một nhà báo. Họ giả định rằng:

- Chính phủ giám sát toàn bộ lưu lượng Internet,
- Hầu hết các Certificate Authority (CA) đã bị xâm nhập,
- Và kẻ tấn công có thể thực hiện Man-in-the-Middle trên mọi kết nối.

i. Hãy thiết kế một giao thức trao đổi khóa phù hợp với tình huống này, chỉ sử dụng những phương tiện mà người dân bình thường có thể tiếp cận (không dùng phần cứng chuyên dụng hay bí mật chia sẻ sẵn).

ii. Phân tích điều gì sẽ xảy ra nếu chính phủ có thể xâm nhập một trong hai thiết bị sau khi quá trình trao đổi khóa đã hoàn tất. Làm thế nào để đảm bảo tính “forward secrecy” (bí mật chuyển tiếp) trong trường hợp này?

2.2 Câu 2. (10 điểm) Thử Thách Thiết Kế Giao Thức

Phân tích cơ chế Trust-on-First-Use (TOFU) trong SSH Tổ chức của bạn muốn triển khai SSH trên hơn 10.000 máy chủ, nhưng mô hình TOFU hiện tại (Trust-On-First-Use) gây ra vấn đề về bảo mật và khả năng sử dụng khi triển khai ở quy mô lớn.

2.2.1 (a) (5 điểm)

Phân tích những kịch bản tấn công cụ thể trong đó mô hình TOFU thất bại trên thực tế. Người dùng dễ bị tấn công nhất trong những tình huống nào?

2.2.2 (b) (5 điểm)

Hãy thiết kế một mô hình xác thực cải tiến cho SSH, vừa giữ được sự đơn giản của TOFU, nhưng lại cung cấp mức bảo mật cao hơn so với TOFU thuần túy.

Trả lời:

(a) Kịch bản tấn công mô hình TOFU

Mô hình Tin tưởng trong lần sử dụng đầu tiên (TOFU) thất bại trong một kịch bản tấn công Man-in-the-Middle (MITM) xảy ra ngay trong lần kết nối đầu tiên.

Kịch bản tấn công cụ thể diễn ra như sau:

Người dùng dễ bị tấn công nhất chính là vào thời điểm họ kết nối với một máy chủ SSH lần đầu tiên, vì đây là lúc khóa máy chủ (host key) chưa được lưu trữ.

1. Kẻ tấn công (Mallory) đã kiểm soát mạng và đứng giữa người dùng (Alice) và máy chủ (Bob).
2. Khi Alice thực hiện kết nối SSH đầu tiên, Mallory chặn yêu cầu này.
3. Mallory mạo danh máy chủ Bob, gửi “khóa máy chủ” (public key) giả mạo của chính hắn cho Alice.
4. Vì đây là lần kết nối đầu tiên, máy khách SSH của Alice sẽ hiển thị thông báo TOFU, hỏi rằng cô có tin tưởng vân tay khóa của Mallory hay không.
5. Alice (người dùng) chấp nhận vân tay khóa này, tin rằng đó là của máy chủ Bob. Khóa giả mạo của Mallory được lưu vào tệp `known_hosts` của Alice.
6. Từ thời điểm này, Mallory thực hiện một kết nối MITM hoàn chỉnh, thiết lập một phiên riêng với Alice (dùng khóa giả mạo) và một phiên riêng với máy chủ Bob (dùng khóa thật của Bob).

Hậu quả: Kẻ tấn công có thể giải mã, đọc và sửa đổi toàn bộ lưu lượng SSH của Alice mà không bị phát hiện. Mọi kết nối sau này của Alice đến máy chủ Bob sẽ tự động tin tưởng khóa của Mallory (vì nó đã được lưu), khiến cuộc tấn công trở nên vô hình.

Trả lời:

(b) Thiết kế mô hình xác thực cải tiến (Mô hình CA nội bộ)

Để giải quyết vấn đề của TOFU ở quy mô lớn (10.000 máy chủ), chúng ta có thể thiết kế một mô hình xác thực dựa trên Chứng chỉ số (Certificates), tương tự như cách TLS/HTTPS hoạt động, nhưng được áp dụng cho SSH.

Thiết kế mô hình “SSH Certificate Authority (CA) nội bộ”:

1. **Tạo CA nội bộ:** Tổ chức sẽ tạo ra một cặp khóa Public/Private cho riêng mình, đóng vai trò là một “Nhà cung cấp chứng chỉ” (CA) nội bộ, đáng tin cậy.
2. **Ký chứng chỉ cho máy chủ:** Thay vì để người dùng tin tưởng 10.000 khóa máy chủ riêng lẻ, CA nội bộ sẽ dùng khóa riêng (private key) của mình để “ký” và cấp phát chứng chỉ cho 10.000 khóa công khai (public host key) của các máy chủ.
3. **Phân phối khóa CA:** Khóa công khai (public key) của CA nội bộ được cài đặt sẵn vào tệp `known_hosts` trên máy của tất cả người dùng (máy khách) và được đánh dấu là một “authority” (nhà chức trách).

Quy trình kết nối:

1. Khi người dùng kết nối đến một máy chủ (ví dụ `server-9876.example.com`) lần đầu tiên, máy chủ đó không chỉ gửi khóa công khai của nó, mà nó gửi chứng chỉ đã được ký bởi CA nội bộ.
2. Máy khách SSH của người dùng tự động kiểm tra chữ ký trên chứng chỉ này.
3. Nếu chữ ký hợp lệ (tức là nó được ký bởi CA nội bộ đã được tin cậy), máy khách sẽ tự động tin tưởng máy chủ và tiến hành kết nối.

So sánh và đánh đổi:

1. Tính đơn giản (Khả năng sử dụng): Đối với người dùng cuối, mô hình này đơn giản hơn TOFU. Họ sẽ không bao giờ nhìn thấy thông báo “Are you sure you want to continue connecting?” đáng sợ nữa. Việc xác thực là tự động và minh bạch, giống như truy cập một trang web HTTPS nội bộ.

2. Bảo mật: Mô hình này an toàn hơn nhiều so với TOFU thuần túy. Lòng tin không được thiết lập “trong” lần kết nối đầu tiên, mà được thiết lập trước đó (khi cài đặt khóa CA). Điều này vô hiệu hóa hoàn toàn kịch bản tấn công MITM trong lần kết nối đầu tiên.

3 Kỹ Thuật Bảo Mật Đường Cong Elliptic (Elliptic Curve Security Engineering)

3.1 Câu 1. (15 điểm) Lựa Chọn Đường Cong Khi Chịu Áp Lực (Curve Selection Under Pressure)

3.1.1 (a) (8 điểm) Tranh Cãi Về "Backdoor" Của Chính Phủ

Bạn là kiến trúc sư bảo mật cho một ứng dụng nhắn tin mới. Các nhà mật mã học đang tranh luận xem liệu NIST P-256 có chứa “backdoor” do chính phủ chèn vào hay không, trong khi Curve25519 cho thấy các thuộc tính an toàn tốt hơn nhưng ít được hỗ trợ phần cứng rộng rãi hơn.

i. Hãy phân tích các mối quan ngại cụ thể đối với các đường cong do NIST quy định:

- Một backdoor có thể trông như thế nào?
- Kẻ tấn công (hoặc chủ thể chèn backdoor) có thể khai thác backdoor đó mà không cần phá vỡ các bài toán nền tảng (ví dụ: DLP/ECDLP) ra sao?

ii. Thiết kế một khung đánh giá rủi ro để chọn giữa P-256 và Curve25519.

- Những yếu tố nào nên ảnh hưởng đến quyết định của bạn? (ví dụ: hiệu năng phần cứng, tính tương thích, mức độ tin cậy của nguồn thuật toán, bảo mật dài hạn, v.v.)

iii. Đối pháp lý báo rằng một số quốc gia yêu cầu tuân thủ chuẩn NIST để bán cho chính phủ.

- Đề xuất một giải pháp vừa xử lý mối lo ngại về backdoor, vừa đảm bảo tuân thủ yêu cầu pháp lý.

Trả lời:

i. Phân tích mối quan ngại về “Backdoor” của NIST

Mối quan ngại về các đường cong do NIST quy định (như P-256) không xuất phát từ một lỗ hổng toán học đã được chứng minh trong chính đường cong, mà xuất phát từ sự thiếu minh bạch trong nguồn gốc của chúng và vụ bê bối liên quan.

Các hằng số “bí ẩn”: Tài liệu chỉ ra rằng các đường cong tiêu chuẩn của NIST (ví dụ: P-256) có các hằng số (như hằng số b trong phương trình) do NSA cung cấp mà không có lời giải thích rõ ràng về nguồn gốc của chúng. Điều này làm dấy lên nghi ngờ rằng các hằng số này có thể đã được lựa chọn đặc biệt để chứa đựng một điểm yếu tiềm ẩn.

Một backdoor có thể trông như thế nào? (Vụ bê bối Dual_EC_DRBG):

Vụ bê bối Dual_EC_DRBG (một bộ tạo số ngẫu nhiên dựa trên ECC) là ví dụ điển hình về một backdoor như vậy.

- Backdoor này bao gồm một cặp điểm P và Q trên đường cong, trong đó NSA biết một số bí mật d sao cho $Q = d \cdot P$.
- Bộ tạo số ngẫu nhiên này được thiết kế để tạo ra các số ngẫu nhiên mà NSA (với việc biết d) có thể dự đoán được.

Cách khai thác (Không phá vỡ ECDLP):

- Kẻ tấn công không cần phá vỡ bài toán ECDLP. Thay vào đó, họ khai thác việc dự đoán được số ngẫu nhiên (nonce).
- Trong các thuật toán như ECDSA (thường dùng với P-256), mỗi chữ ký yêu cầu một số ngẫu nhiên bí mật k (nonce).

- Nếu kẻ tấn công (NSA) có thể dự đoán được nonce k này (thông qua backdoor trong bộ tạo số ngẫu nhiên như Dual_EC_DRBG), họ có thể dễ dàng tính toán ra khóa riêng (private key) d của người dùng. Đây chính xác là lỗ hổng đã xảy ra trong vụ hack Sony PS3, nơi k bị sử dụng lại.

ii. Khung đánh giá rủi ro (P-256 vs. Curve25519)

Khi thiết kế khung đánh giá rủi ro cho một ứng dụng nhắn tin mới, chúng ta cần cân nhắc các yếu tố sau:

Yếu tố	NIST P-256	Curve25519	Nguồn tài liệu
Mức độ tin cậy (Nguồn gốc)	Rủi ro cao. Các hằng số “bí ẩn” từ NSA. Lòng tin bị xói mòn nghiêm trọng sau vụ Dual_EC_DRBG.	Rủi ro thấp. Được thiết kế bởi Daniel J. Bernstein (DJB) với triết lý “tham số minh bạch” và “không có gì giấu giếm”.	
Bảo mật triển khai	Rủi ro cao. Dễ bị tấn công (ví dụ: Invalid Curve Attacks, Small Subgroup Attacks) nếu không kiểm tra (validate) điểm cẩn thận.	Rủi ro thấp. Được thiết kế để “an toàn hơn khi triển khai”, giúp giảm thiểu rủi ro từ các lỗi triển khai phổ biến.	
Hiệu năng	Hiệu năng tốt, thường được hỗ trợ bởi phần cứng.	Ưu việt hơn. “Nhanh hơn” P-256. Lợi ích lớn về tốc độ và “giảm tiêu thụ pin”, rất quan trọng cho ứng dụng nhắn tin di động.	
Hệ sinh thái (Tính tương thích)	Được chuẩn hóa bởi NIST. Bắt buộc cho tuân thủ chính phủ (theo đề bài).	Rất mạnh. Đã trở thành tiêu chuẩn và được sử dụng rộng rãi trong: Signal, WhatsApp, TLS 1.3, OpenSSH. Đây là yếu tố then chốt cho một ứng dụng nhắn tin.	

Quyết định: Đối với một ứng dụng nhắn tin ưu tiên bảo mật và quyền riêng tư, Curve25519 là lựa chọn vượt trội rõ rệt do có các tham số minh bạch, an toàn hơn khi triển khai và hiệu năng cao.

iii. Giải pháp cân bằng giữa tuân thủ và bảo mật

Để vừa tuân thủ yêu cầu pháp lý (sử dụng chuẩn NIST) vừa xử lý mối lo ngại về backdoor (liên quan đến việc dự đoán nonce), giải pháp là sử dụng đường cong P-256 nhưng loại bỏ sự phụ thuộc vào số ngẫu nhiên không đáng tin cậy.

1. Tuân thủ đường cong: Sử dụng đường cong NIST P-256 cho các hoạt động mã hóa (ví dụ: ECDH) và chữ ký (ECDSA) để đáp ứng yêu cầu pháp lý.

2. Xử lý mối lo ngại về Backdoor (Nonce):

- Mối lo ngại chính là việc backdoor (như Dual_EC_DRBG) cho phép kẻ tấn công dự đoán số ngẫu nhiên k (nonce) dùng trong chữ ký ECDSA.
- Giải pháp là sử dụng chữ ký ECDSA với nonce xác định (deterministic nonce) (theo chuẩn RFC 6979).
- Cơ chế này vay mượn ý tưởng cốt lõi của EdDSA: thay vì tạo k từ một bộ tạo số ngẫu nhiên (có thể bị chèn backdoor), k (hoặc r trong EdDSA) được tạo ra bằng cách hash (băm) khóa riêng và tin nhắn.
- Bằng cách này, chúng ta “loại bỏ hoàn toàn nhóm lỗ hổng liên quan đến ‘randomness’”.

Kết luận: Bằng cách sử dụng P-256 (tuân thủ) kết hợp với nonce xác định (an toàn), chúng ta đáp ứng yêu cầu của chính phủ mà không cần phải tin tưởng vào các bộ tạo số ngẫu nhiên do NIST cung cấp.

3.1.2 (b) (7 điểm) Kịch Bản Tấn Công "Invalid Curve"

Một nhà nghiên cứu bảo mật phát hiện rằng triển khai ECDH của bạn không kiểm tra điểm đầu vào, làm cho nó dễ bị tấn công invalid-curve.

i. Hãy thiết kế một tấn công cụ thể lợi dụng lỗ hổng này.

- Kẻ tấn công có thể trích xuất thông tin gì từ phía nạn nhân (ví dụ: khóa riêng, các bit của khóa, hay phép đo khác)?

ii. Phát triển một chiến lược kiểm tra và xác thực input toàn diện để ngăn chặn lớp tấn công này.

- Giải pháp của bạn có ảnh hưởng hiệu năng như thế nào?

Trả lời:

i. Thiết kế tấn công "Invalid Curve"

Cuộc tấn công này lợi dụng việc nạn nhân (Bob) không kiểm tra xem điểm công khai (public key) nhận được có thực sự nằm trên đường cong tiêu chuẩn đã thống nhất hay không.

Các bước tấn công:

1. **Thiết lập của kẻ tấn công (Alice):** Kẻ tấn công chọn một đường cong hoàn toàn khác (ví dụ: $y^2 = x^3 + ax + b'$). Đường cong này được lựa chọn đặc biệt vì nó là một đường cong yếu mà kẻ tấn công biết cách giải bài toán ECDLP một cách dễ dàng.
2. **Gửi điểm giả mạo:** Kẻ tấn công gửi một điểm A nằm trên đường cong yếu này cho nạn nhân Bob.
3. **Tính toán của nạn nhân (Bob):** Bob nhận điểm A và (do thiếu kiểm tra) tiến hành tính toán khóa chung $S = b \cdot A$ (với b là khóa riêng của Bob).
4. **Lỗ hổng bị khai thác:** Các công thức toán học dùng để "cộng" điểm (phép toán cơ bản của $b \cdot A$) thường không sử dụng hằng số b của phương trình đường cong. Do đó, Bob thực hiện phép tính của mình trên đường cong yếu của kẻ tấn công mà không gặp bất kỳ lỗi nào.

Thông tin bị trích xuất:

Khóa chung S mà Bob tính ra sẽ nằm trên đường cong yếu của kẻ tấn công. Vì kẻ tấn công biết cách giải bài toán ECDLP trên đường cong này, chúng không chỉ tìm ra được khóa chung S mà còn có thể suy ra toàn bộ khóa riêng b của nạn nhân.

ii. Chiến lược xác thực input (Validation)

Một chiến lược xác thực toàn diện phải bảo vệ chống lại cả hai cuộc tấn công dựa trên điểm đầu vào được đề cập trong tài liệu: "Invalid Curve" và "Small Subgroup".

Các bước kiểm tra bắt buộc:

1. **Kiểm tra phương trình đường cong (Chống Invalid Curve):** Khi nhận được một điểm $A(x, y)$, máy chủ phải xác minh rằng các tọa độ này thực sự thỏa mãn phương trình đường cong tiêu chuẩn đã thống nhất: $y^2 = x^3 + ax + b \pmod{p}$. Nếu phương trình không cân bằng, điểm đó phải bị từ chối ngay lập tức.
2. **Kiểm tra phân nhóm (Chống Small Subgroup):** Sau khi xác nhận điểm nằm trên đường cong, máy chủ phải kiểm tra xem điểm đó có thuộc phân nhóm an toàn (nhóm lớn, có bậc l) hay không. Điều này được thực hiện bằng cách tính $l \cdot A$. Nếu kết quả là điểm vô cực ($\mathcal{O}$), thì điểm đó an toàn; nếu không, nó thuộc một phân nhóm nhỏ và phải bị từ chối.

Ảnh hưởng hiệu năng:

- Việc thực hiện các bước xác thực này, đặc biệt là bước 2 (Kiểm tra phân nhóm, vốn là một phép nhân vô hướng $l \cdot A$), sẽ làm tăng thêm chi phí tính toán cho mỗi lần trao đổi khóa.

- Tài liệu không định lượng chi phí này, nhưng nhấn mạnh rằng các đường cong hiện đại như Curve25519 được ưu tiên sử dụng vì chúng vừa “nhanh hơn” vừa “an toàn hơn khi triển khai”. Thiết kế của Curve25519 vốn đã giảm thiểu các rủi ro tấn công này, làm cho việc xác thực trở nên hiệu quả hơn (hoặc trong một số trường hợp, không cần thiết) so với các đường cong NIST cũ.

3.2 Câu 2. (15 điểm) Phân Tích Lỗ Hổng Triển Khai (Implementation Vulnerability Analysis)

3.2.1 (a) (8 điểm) Phòng Thí Nghiệm Tấn Công Kênh Bên (Side-Channel Attack Laboratory)

Bạn được giao nhiệm vụ kiểm thử triển khai ECDSA trên một thiết bị nhúng để tìm lỗ hổng kênh bên.

i. Hãy thiết kế một tấn công timing chống ECDSA.

- Bạn sẽ cố gắng phân biệt giữa hai phép toán nào? (ví dụ: thao tác nhân điểm vs. thao tác bổ sung, hoặc xử lý trường hợp bù trừ khác nhau, v.v.)
- ii. Phát triển các biện pháp khắc phục vừa giữ được hiệu năng, vừa chống lại tấn công của bạn.
- Những kỹ thuật “constant-time” cụ thể nào bạn sẽ áp dụng?

Trả lời:

i. Thiết kế tấn công timing chống ECDSA

A. Kiến thức nền về ECDSA

Quy trình ký ECDSA (đơn giản hóa):

1. Chọn nonce ngẫu nhiên: k
2. Tính điểm: $R = k \cdot G$
3. Lấy tọa độ x : $r = R.x \pmod{n}$
4. Tính: $s = k^{-1} \cdot (H(m) + r \cdot d) \pmod{n}$
5. Chữ ký: (r, s)

B. Điểm yếu có thể khai thác

Các phép toán có thời gian khác nhau:

1. Scalar multiplication: $k \cdot G$

- Phụ thuộc vào các bit của k
- Có thể phân biệt bit 0 vs bit 1

2. Modular inversion: k^{-1}

- Có thể phụ thuộc vào giá trị k
- Một số thuật toán không constant-time

3. Conditional branches trong point addition

- Trường hợp đặc biệt: điểm tại vô cực, điểm đối nhau
- Tạo ra timing leakage

C. Thiết kế tấn công timing

Mục tiêu: Khôi phục nonce $k \rightarrow$ Tính được private key d

Tấn công 1: Timing leak trong Scalar Multiplication

Bước 1: Hiểu thuật toán scalar multiplication

Thuật toán “Double-and-Add” (không an toàn):

```
def scalar_mult_naive(k, G):  
    """  
    Tính k*G bằng double-and-add  
    KHÔNG AN TOÀN - có timing leak  
    """  
    result = POINT_AT_INFINITY  
    temp = G  
  
    # Duyệt từng bit của k (từ LSB đến MSB)  
    for bit in bits(k):  
        if bit == 1:          # <- ĐIỀU KIỆN RÕ RÀNG  
            result = result + temp  # Phép cộng điểm (chạm)  
            temp = temp + temp      # Phép double (luôn thực hiện)  
  
    return result
```

Vấn đề:

- Khi bit = 1: thực hiện cả DOUBLE và ADD
- Khi bit = 0: chỉ thực hiện DOUBLE
- Timing khác nhau → Rò rỉ thông tin về các bit của k

Bước 2: Thu thập timing measurements

Setup thí nghiệm:

```
import time  
import statistics  
  
def measure_sign_timing(message, device):  
    """  
    Đo thời gian ký cho một message  
    """  
    timings = []  
  
    for i in range(1000): # Thu thập 1000 mẫu  
        start = time.perf_counter()  
        signature = device.sign_ecdsa(message)  
        end = time.perf_counter()  
  
        timings.append(end - start)  
  
    return {  
        'mean': statistics.mean(timings),  
        'stdev': statistics.stdev(timings),  
        'samples': timings  
    }
```

Bước 3: Phân tích timing để trích xuất bit của k

Phương pháp:

```
def analyze_timing_leakage(timings, threshold):  
    """  
    Phan tich timing de doan cac bit cua k  
  
    Gia dinh:  
    - Timing cao -> nhieu bit 1 trong k  
    - Timing thap -> it bit 1 trong k  
    """  
    bit_guesses = []  
  
    for t in timings:  
        if t > threshold:  
            bit_guesses.append(1) # Bit co the la 1  
        else:  
            bit_guesses.append(0) # Bit co the la 0  
  
    return bit_guesses
```

Bước 4: Khôi phục nonce k

Khi có đủ thông tin về các bit của k :

```
def recover_nonce_from_bits(bit_pattern, signature, message, pubkey):  
    """  
    Voi mot so bit cua k da biet, brute-force cac bit con lai  
    """  
    r, s = signature  
    h = hash(message)  
  
    # Gia su biet duoc 200 bit tren 256 bit cua k  
    known_bits = bit_pattern # 200 bit da biet  
    unknown_bits = 56        # 56 bit chua biet  
  
    # Brute-force  $2^{56}$  kha nang (kha thi!)  
    for candidate in range(2**unknown_bits):  
        k_guess = construct_k(known_bits, candidate)  
  
        # Kiem tra:  $r == (k \cdot G).x$  ?  
        R = k_guess * G  
        if R.x == r:  
            return k_guess  
  
    return None
```

Bước 5: Tính private key d từ k

Công thức ECDSA:

$$s = k^{-1} \cdot (h + r \cdot d) \pmod{n}$$
$$\rightarrow d = r^{-1} \cdot (s \cdot k - h) \pmod{n}$$

```
def recover_private_key(k, signature, message):  
    """
```

```
Khi da biet k, tinh private key d
"""
r, s = signature
h = hash(message)
n = CURVE_ORDER

# d = r^(-1)*(s*k - h) (mod n)
d = (mod_inverse(r, n) * (s * k - h)) % n

return d
```

Tấn công 2: Timing leak trong Modular Inversion

Phân biệt phép toán k^{-1}

Thuật toán Extended Euclidean (không constant-time):

```
def mod_inverse_unsafe(a, m):
    """
    KHONG AN TOAN - so vong lap phu thuoc vao gia tri a
    """
    old_r, r = a, m
    old_s, s = 1, 0

    # So vong lap phu thuoc vao gcd(a, m)
    while r != 0:          # <- So lan lap khong co dinh
        quotient = old_r // r
        old_r, r = r, old_r - quotient * r
        old_s, s = s, old_s - quotient * s

    return old_s % m
```

Timing leak:

- k nhỏ $\rightarrow$ ít vòng lặp $\rightarrow$ nhanh
- k lớn $\rightarrow$ nhiều vòng lặp $\rightarrow$ chậm

Khai thác:

```
def attack_modular_inverse_timing(device):
    """
    Do timing de doan kich thuoc tuong doi cua k
    """
    timings = []

    for i in range(10000):
        t = measure_sign_timing(f"message_{i}", device)
        timings.append(t)

    # Phan loai k nho vs k lon dua tren timing
    small_k_signatures = [sig for sig, t in timings if t < threshold]

    # Tan cong nhung chu ky co k nho (de brute-force hon)
    return small_k_signatures
```

Tấn công 3: Branch Prediction Attack

Phân biệt các trường hợp đặc biệt trong point addition

Point addition với branches:

```
def point_add_unsafe(P, Q):  
    """  
    KHONG AN TOAN - co conditional branches  
    """  
    # Truong hop dac biet 1: P = 0 (diem vo cuc)  
    if P == POINT_AT_INFINITY:    # <- Branch 1  
        return Q  
  
    # Truong hop dac biet 2: Q = 0  
    if Q == POINT_AT_INFINITY:    # <- Branch 2  
        return P  
  
    # Truong hop dac biet 3: P = Q  
    if P == Q:                    # <- Branch 3  
        return point_double(P)  
  
    # Truong hop dac biet 4: P = -Q  
    if P.x == Q.x and P.y == -Q.y: # <- Branch 4  
        return POINT_AT_INFINITY  
  
    # Truong hop thong thuong  
    return general_point_add(P, Q)
```

Timing leak:

- Các trường hợp đặc biệt có timing khác nhau
- Rò rỉ thông tin về cấu trúc của k

D. Kết quả tổng hợp

Kịch bản tấn công hoàn chỉnh:

1. Thu thập 10,000+ chữ ký với timing measurements
2. Phân tích timing $\rightarrow$ Đoán được $\sim 200/256$ bit của k
3. Brute-force 56 bit còn lại ($2^{56} \approx 1$ giờ trên laptop)
4. Verify k bằng cách kiểm tra $r = (k \cdot G).x$
5. Tính private key: $d = r^{-1} \cdot (s \cdot k - h)$
6. **Game over** - Có thể ký bất kỳ message nào!

Trả lời:

ii. Biện pháp khắc phục constant-time

A. Nguyên tắc cơ bản

Mục tiêu: Đảm bảo timing **không** phụ thuộc vào dữ liệu bí mật

3 nguyên tắc vàng:

- **Không** có conditional branches phụ thuộc vào secret
- **Không** có memory access patterns phụ thuộc vào secret
- Mọi đường thực thi phải mất cùng thời gian

B. Khắc phục 1: Constant-time Scalar Multiplication

Thuật toán Montgomery Ladder (constant-time):

```
def scalar_mult_constant_time(k, G):  
    """  
    AN TOAN - constant-time scalar multiplication  
    """  
    R0 = POINT_AT_INFINITY  
    R1 = G  
  
    # Duyệt từng bit của k (từ MSB đến LSB)  
    for i in range(255, -1, -1):  
        bit = (k >> i) & 1  
  
        # LUÔN thực hiện CẢ HAI phép toán  
        # Không có branch conditional  
        R0, R1 = cswap(bit, R0, R1) # Conditional swap  
        R1 = R0 + R1                 # Add  
        R0 = R0 + R0                 # Double  
        R0, R1 = cswap(bit, R0, R1) # Swap back  
  
    return R0
```

Constant-time conditional swap:

```
def cswap(condition, a, b):  
    """  
    Swap a và b nếu condition = 1, giữ nguyên nếu condition = 0  
    Không có branch, chỉ dùng bitwise operations  
    """  
    # Tạo mask: 0 hoặc 0xFFFFFFFF  
    mask = -condition # 0 -> 0x00000000, 1 -> 0xFFFFFFFF  
  
    # XOR swap với mask  
    temp = mask & (a ^ b)  
    a = a ^ temp  
    b = b ^ temp  
  
    return a, b
```

Lợi ích:

- Timing hoàn toàn độc lập với giá trị k
- Luôn thực hiện cùng số phép toán
- Hiệu năng tốt (overhead $\sim 10\text{-}20\%$)

C. Khắc phục 2: Constant-time Modular Inversion

Sử dụng Fermat's Little Theorem:

```
def mod_inverse_constant_time(a, p):  
    """  
    AN TOAN - constant-time inversion  
    Su dung:  $a^{-1} = a^{(p-2)} \pmod p$   
    """  
    # Modular exponentiation voi square-and-multiply constant-time  
    return mod_exp_constant_time(a, p - 2, p)  
  
def mod_exp_constant_time(base, exp, mod):  
    """  
    Tinh  $base^{exp} \pmod{mod}$  theo constant-time  
    """  
    result = 1  
    base = base % mod  
  
    # LUON duyet tat ca 256 bit  
    for i in range(255, -1, -1):  
        result = (result * result) % mod # Luon square  
  
        # Conditional multiply (constant-time)  
        bit = (exp >> i) & 1  
        mask = -bit  
        temp = (result * base) % mod  
        result = select(mask, temp, result) # Chon khong dung branch  
  
    return result  
  
def select(mask, a, b):  
    """  
    Tra ve a neu mask = 0xFFFFFFFF, b neu mask = 0x00000000  
    Khong co branch  
    """  
    return (mask & a) | (~mask & b)
```

Lợi ích:

- Thời gian cố định bất kể giá trị a
- Không rò rỉ thông tin qua timing
- Hiệu năng: ~ 256 phép squaring + ~ 128 phép multiply (trung bình)

D. Khắc phục 3: Constant-time Point Operations

Point addition không có branches:

```
def point_add_constant_time(P, Q):  
    """  
    AN TOAN - complete point addition formula  
    Hoat dong dung cho MOI truong hop ma khong can branch
```



```
"""
# Su dung complete formulas tu Renes-Costello-Batina (2016)
# Cac cong thuc nay xu ly TAT CA truong hop dac biet
# ma khong can if-else

X1, Y1, Z1 = P
X2, Y2, Z2 = Q

# 12 phep nhan field (co dinh)
t0 = X1 * X2
t1 = Y1 * Y2
t2 = Z1 * Z2
t3 = X1 + Y1
t4 = X2 + Y2
t3 = t3 * t4
t4 = t0 + t1
# ... (cac buoc con lai)

# Khong co conditional branch nao
# Luon thuc hien DUNG 12 phep nhan

return (X3, Y3, Z3)
```

Curve25519 - Được thiết kế cho constant-time:

Đặc điểm:

- Complete addition formulas (không cần trường hợp đặc biệt)
- Montgomery curve (phép toán x-coordinate only)
- Thiết kế sẵn cho constant-time implementation

```
# Curve25519 x-coordinate only ladder
def curve25519_scalar_mult(k, u):
    """
    Đơn giản, nhanh, constant-time
    """
    x_1 = u
    x_2 = 1
    z_2 = 0
    x_3 = u
    z_3 = 1

    for i in range(254, -1, -1):
        bit = (k >> i) & 1
        x_2, x_3 = cswap(bit, x_2, x_3)
        z_2, z_3 = cswap(bit, z_2, z_3)

        # Montgomery differential addition (5M + 4S + 1*a24)
        # Constant-time, không có branches
        A = x_2 + z_2
```

```
AA = A^2
# ... (cac buoc con lai)

x_2, x_3 = cswap(bit, x_2, x_3)
z_2, z_3 = cswap(bit, z_2, z_3)

return x_2 / z_2
```

E. Khắc phục 4: Sử dụng EdDSA thay vì ECDSA

EdDSA deterministic nonce:

```
def ed25519_sign(private_key, message):
    """
    AN TOAN - EdDSA không cần random nonce
    """
    # Hash private key
    h = SHA512(private_key)
    a = h[:32] # Scalar
    prefix = h[32:]

    # Nonce được tính deterministic
    r = SHA512(prefix || message)
    r = r mod L # L là order của curve

    # Không có random -> Không có timing leak liên quan đến RNG!
    R = r * G

    hram = SHA512(R || public_key || message)
    S = (r + hram * a) mod L

    return (R, S)
```

Lợi ích:

- Loại bỏ hoàn toàn lỗ hổng liên quan đến nonce k
- Không cần RNG an toàn
- Deterministic → Dễ test
- Constant-time implementations có sẵn (libsodium, etc.)

F. Biện pháp bổ sung

1. Blinding (Làm mù)

```
def scalar_mult_with_blinding(k, G):
    """
    Thêm random vào k để che giấu giá trị thực
    """
    # Chọn random blind r
    r = random_scalar()
    n = CURVE_ORDER
```

```
# k' = k + r*n (mod n) = k (mod n)
k_blinded = (k + r * n) % n

# Tính voi k', nhưng kết quả giống k*G
result = scalar_mult_constant_time(k_blinded, G)

return result
```

2. Randomized Projective Coordinates

```
def randomize_point(P):
    """
    Biểu diễn cùng một điểm với tọa độ khác nhau
    """
    X, Y, Z = P
    lambda_rand = random_nonzero()

    # (X:Y:Z) và (lambda*X:lambda*Y:lambda*Z) biểu diễn cùng điểm
    return (lambda_rand * X, lambda_rand * Y, lambda_rand * Z)
```

So sánh hiệu năng

Kỹ thuật	Overhead	Độ an toàn	Khuyến nghị
Naive implementation	0%	Không an toàn	Không bao giờ dùng
Montgomery Ladder	+15%	An toàn	Khuyến nghị
Complete formulas	+20%	Rất an toàn	Khuyến nghị
EdDSA (Ed25519)	+10%	Rất an toàn	Tốt nhất
Blinding	+30%	Defense-in-depth	Bổ sung

Khuyến nghị cuối cùng

Nên:

- Dùng Ed25519 thay vì ECDSA cho triển khai mới
- Sử dụng Montgomery Ladder hoặc complete formulas
- Dùng thư viện đã được kiểm định (libsodium, BoringSSL)
- Test với tools như `dudect` để verify constant-time
- Kết hợp nhiều lớp bảo vệ (blinding + constant-time)

Không:

- Tự implement từ đầu
- Dùng branches phụ thuộc secret data
- Tin rằng “compiler sẽ tối ưu đúng”
- Bỏ qua timing attacks vì “quá khó khai thác”

Nguồn tham chiếu:

- Mục 2.3 (Chương 8): ECDSA và EdDSA
- Mục 3.2 (Chương 8): Curve25519 an toàn hơn
- Ví dụ 2 (Chương 8): PlayStation 3 hack - bài học về ECDSA

3.2.2 (b) (7 điểm) Khủng Hoảng Xác Thực Ed25519 (Ed25519 Validation Crisis)

Bạn phát hiện hai thư viện Ed25519 phổ biến chấp nhận các chữ ký khác nhau như hợp lệ cho cùng một thông điệp và khóa công khai — dẫn đến mất khả năng tương tác.

i. Khảo sát nguyên nhân của sự không nhất quán này:

- Những bước kiểm tra nào ở các triển khai khác nhau có thể không xử lý giống nhau?

ii. Phân tích hệ quả an ninh:

- Liệu kẻ tấn công có thể lợi dụng những khác biệt đó để tạo ra tấn công thực tế (ví dụ: cho phép ký giả mạo hoặc tận dụng để né kiểm tra xác thực)?

Trả lời:

i. Khảo sát nguyên nhân của sự không nhất quán

Vấn đề: Hai thư viện Ed25519 khác nhau có thể cho kết quả VALID và INVALID trên cùng một chữ ký.

Nguyên nhân: Một số thư viện “lỏng lẻo” (lenient) đã bỏ qua các bước kiểm tra (validation) quan trọng được quy định trong chuẩn RFC 8032 để tối ưu hóa tốc độ.

Các bước bị bỏ qua:

- **Không kiểm tra $S < L$:** Không kiểm tra S có nằm trong phạm vi chuẩn hay không.
- **Không kiểm tra điểm trên đường cong:** Cho phép các điểm từ đường cong khác (yếu hơn).
- **Không kiểm tra Small Subgroup:** Không kiểm tra các điểm có bậc (order) nhỏ.
- **Không kiểm tra Canonical Encoding:** Chấp nhận nhiều cách mã hóa (encoding) cho cùng một điểm.

ii. Phân tích hệ quả an ninh

1. Signature Malleability (Khi bỏ qua $S < L$):

Kẻ tấn công có thể thay đổi S thành $S + L$. Chữ ký $(R, S + L)$ mới vẫn được thư viện lỏng lẻo chấp nhận là hợp lệ.

Hậu quả: Hai chữ ký hợp lệ khác nhau cho cùng một giao dịch, dẫn đến hai Transaction ID khác nhau. Trong blockchain, điều này gây ra Consensus Failure (mạng bị chia rẽ).

2. Giả mạo chữ ký (Khi bỏ qua kiểm tra Curve/Subgroup):

Invalid Curve Attack: Kẻ tấn công gửi một điểm trên một đường cong yếu mà chúng biết cách giải. Nếu thư viện bỏ qua kiểm tra, kẻ tấn công có thể giả mạo chữ ký và thậm chí khôi phục khóa riêng của nạn nhân.

Small Subgroup Attack: Cho phép kẻ tấn công thực hiện các cuộc tấn công giới hạn trong các phân nhóm nhỏ.

Khuyến nghị khắc phục:

- **Tuân thủ nghiêm ngặt RFC 8032:** Thực hiện tất cả các bước kiểm tra, không bỏ sót.
- **Sử dụng thư viện đã được kiểm định:** Dùng các thư viện uy tín như libsodium.
- **Kiểm tra tương thích chéo:** Đảm bảo các thư viện khác nhau trong hệ thống đều cho cùng một kết quả.

4 Case Studies Mật Mã Ứng Dụng (Applied Cryptography Case Studies)

4.1 Câu 1. (15 điểm) Phân Tích Cơ Chế Chữ Ký Cho Tiền Điện Tử

Một dự án tiền điện tử mới đang lựa chọn giữa ECDSA và Ed25519 cho chữ ký giao dịch. Yêu cầu hệ thống bao gồm:

- Thông lượng giao dịch cao (hàng ngàn chữ ký mỗi giây).
- Bảo mật lâu dài (hệ thống cần giữ an toàn trong nhiều thập kỷ).
- Tương thích với ví phần cứng và thiết bị di động.
- Ký định tính (deterministic signing) để đảm bảo khả năng tái lập lại.

Hãy phân tích quyết định này:

4.1.1 (a) (5 điểm) So Sánh ECDSA và Ed25519

So sánh ECDSA và Ed25519 cho từng yêu cầu nêu trên. Thuật toán nào đáp ứng tốt hơn từng tiêu chí và vì sao?

4.1.2 (b) (5 điểm) Thảo Luận Về Hệ Quả Của Tính Có Thể Thay Đổi Chữ Ký (Signature Malleability)

Thảo luận về hệ quả của tính có thể thay đổi chữ ký (signature malleability). Điều này ảnh hưởng đến từng thuật toán như thế nào và tại sao có thể quan trọng với ứng dụng tiền điện tử?

4.1.3 (c) (5 điểm) Đưa Ra Khuyến Nghị Cuối Cùng

Đưa ra khuyến nghị cuối cùng cùng lý do, cân nhắc cả khía cạnh kỹ thuật và thực tiễn (practical factors).

Trả lời:

(a) So sánh ECDSA và Ed25519

Dưới đây là so sánh hai thuật toán dựa trên các yêu cầu của dự án tiền điện tử:

1. Thông lượng giao dịch cao (Hiệu năng)

Ed25519 (EdDSA): Đáp ứng tốt hơn. Tài liệu mô tả EdDSA là “nhanh hơn” so với các tiêu chuẩn cũ. Ed25519 sử dụng Curve25519, vốn được biết là “nhanh hơn” các đường cong NIST (thường dùng với ECDSA). Tốc độ tính toán nhanh hơn đồng nghĩa với thông lượng chữ ký cao hơn.

ECDSA: Mặc dù nhanh hơn RSA, nhưng thường chậm hơn EdDSA.

2. Bảo mật lâu dài

Ed25519 (EdDSA): Đáp ứng tốt hơn. EdDSA được mô tả là “an toàn hơn”. Nó được thiết kế để “loại bỏ hoàn toàn nhóm lỗ hổng liên quan đến ‘randomness’”.

ECDSA: Rủi ro cao hơn. ECDSA có một “điểm yếu chí mạng” (critical weakness). Nó yêu cầu một số ngẫu nhiên (nonce) k bí mật cho mỗi chữ ký. Nếu k bị rò rỉ, bị lặp lại, hoặc có thể dự đoán, “khóa riêng (private key) d sẽ bị lộ ngay lập tức”. Vụ hack Sony PS3 là một ví dụ kinh điển về lỗi này. Đây là một rủi ro nghiêm trọng về lâu dài.

3. Tương thích với ví phần cứng và thiết bị di động

Cả hai: Đều đáp ứng tốt. Cả hai đều là thuật toán Mật mã Đường cong Elliptic (ECC), vốn cung cấp khóa nhỏ hơn với độ an toàn tương đương. Điều này giúp “giảm tiêu thụ pin” và “tiết kiệm băng thông”, khiến chúng “đặc biệt quan trọng cho thiết bị di động và IoT”.

Thực tế: ECDSA đã được sử dụng rộng rãi trong Bitcoin và Ethereum, trong khi Ed25519 được dùng trong Signal và WhatsApp.

4. Ký định tính (Deterministic signing)

Ed25519 (EdDSA): Đáp ứng hoàn hảo. EdDSA là “xác định” (deterministic). Nó “không cần số k ngẫu nhiên”. Thay vào đó, nó tạo ra nonce bằng cách băm (hash) khóa riêng và tin nhắn.

ECDSA: Không đáp ứng (theo thiết kế cơ bản). Thiết kế tiêu chuẩn của ECDSA yêu cầu một số k ngẫu nhiên, khiến nó vốn không có tính định tính. (Mặc dù các triển khai sau này (như RFC 6979) đã thêm tính năng định tính để khắc phục điểm yếu, nhưng thiết kế gốc của EdDSA đã giải quyết việc này).

Trả lời:

(b) Hệ quả của tính có thể thay đổi chữ ký (Signature Malleability)

Sau khi rà soát kỹ lưỡng cả hai tài liệu được cung cấp (1.7-hard-problems&diffie-hellman.md và 1.8-elliptic_curves&digital_signatures.md), tôi xác nhận rằng chủ đề về “tính có thể thay đổi chữ ký” (signature malleability) không được đề cập đến trong bất kỳ phần nào của tài liệu.

Do đó, tôi không thể phân tích hệ quả của nó hoặc ảnh hưởng của nó đến hai thuật toán dựa trên nguồn tài liệu đã cung cấp.

Trả lời:

(c) Đưa ra khuyến nghị cuối cùng

Khuyến nghị: Sử dụng Ed25519.

Lý do:

1. **Bảo mật vượt trội:** Ed25519 giải quyết được “điểm yếu chí mạng” của ECDSA liên quan đến việc tạo số ngẫu nhiên k . Bằng cách sử dụng chữ ký xác định, nó loại bỏ hoàn toàn nguy cơ lộ khóa riêng do lỗi triển khai nonce, điều đã từng xảy ra trong thực tế (ví dụ Sony PS3). Đây là yếu tố quan trọng nhất cho “bảo mật lâu dài”.
2. **Đáp ứng yêu cầu ký định tính:** Ed25519 được thiết kế “xác định” ngay từ đầu, đáp ứng trực tiếp yêu cầu của dự án.
3. **Hiệu năng cao:** Ed25519 “nhanh hơn”, đáp ứng yêu cầu về “thông lượng giao dịch cao”.
4. **Phù hợp với di động:** Giống như các thuật toán ECC khác, nó rất phù hợp cho thiết bị di động và đã được chứng minh trong các ứng dụng quy mô lớn như Signal và WhatsApp.

Tài liệu đã kết luận rõ ràng: “Khuyến nghị: Sử dụng Ed25519 cho các dự án mới”. Mặc dù ECDSA đã được sử dụng trong các tiền điện tử lớn, Ed25519 là một tiêu chuẩn hiện đại, an toàn hơn và nhanh hơn, phù hợp hơn cho một dự án mới.

4.2 Câu 2. (15 điểm) Kiến Trúc Hệ Thống Liên Lạc An Toàn

Bạn đang thiết kế hệ thống liên lạc an toàn cho một tổ chức lớn (10.000+ nhân viên) cần bảo vệ khỏi cả tấn công bên ngoài và nguy cơ nội bộ. Hệ thống phải hỗ trợ nhắn tin thời gian thực, chia sẻ file và gọi thoại. Hãy thiết kế và phân tích một giải pháp hoàn chỉnh:

4.2.1 (a) (5 điểm) Chỉ Định Các Lựa Chọn Thuật Toán Mật Mã

Chỉ định các lựa chọn thuật toán mật mã cho:

- Giao thức trao đổi khóa (Key exchange protocols)
- Thuật toán chữ ký số (Digital signature schemes)
- Thuật toán mã đối xứng (Symmetric encryption algorithms)
- Hàm băm và MACs (Hash functions and MACs).

4.2.2 (b) (5 điểm) Mô Tả Kiến Trúc Quản Lý Khóa

Mô tả kiến trúc quản lý khóa của bạn. Làm sao bạn khởi tạo lòng tin (bootstrap trust), phân phối khóa, và xử lý luân chuyển (key rotation)?

4.2.3 (c) (5 điểm) Phân Tích Tính Bảo Mật Của Hệ Thống

Phân tích tính bảo mật của hệ thống trước các kịch bản tấn công:

- Nghe lén mạng (network eavesdropping)
- Máy chủ bị xâm phạm (server compromise)
- Thiết bị đầu cuối bị xâm phạm (endpoint compromise)
- Tấn công từ nội bộ (insider attacks).

Trả lời:

(a) Chỉ định các lựa chọn thuật toán mật mã

Dựa trên các khuyến nghị và ví dụ trong tài liệu, các thuật toán sau được lựa chọn:

1. **Giao thức trao đổi khóa (Key exchange protocols):** ECDH (Elliptic Curve Diffie-Hellman). Đây là phiên bản hiện đại của Diffie-Hellman, cung cấp hiệu quả vượt trội (khóa nhỏ hơn). Cụ thể, chúng ta nên dùng triển khai dựa trên Curve25519 vì các tham số minh bạch và thiết kế an toàn của nó.
2. **Thuật toán chữ ký số (Digital signature schemes):** Ed25519 (EdDSA). Tài liệu khuyến nghị EdDSA/Ed25519 cho các dự án mới vì nó “an toàn và nhanh hơn”. Quan trọng nhất, nó là “xác định” (deterministic) và “loại bỏ hoàn toàn nhóm lỗ hổng liên quan đến ‘randomness’” mà ECDSA mắc phải.
3. **Thuật toán mã đối xứng (Symmetric encryption algorithms):** AES hoặc ChaCha20. Khóa cho các thuật toán này sẽ được tạo ra từ khóa chung bí mật (S) thu được từ ECDH.
4. **Hàm băm và MACs (Hash functions and MACs):** Tài liệu đề cập đến việc sử dụng các hàm HASH như một thành phần cốt lõi của chữ ký (ví dụ: h là hash của tin nhắn trong ECDSA, và EdDSA tạo nonce bằng cách HASH(khóa riêng || tin nhắn)). Mặc dù tài liệu không chỉ định một thuật toán HASH cụ thể, chúng là bắt buộc để đảm bảo tính toàn vẹn của tin nhắn và cho hoạt động an toàn của EdDSA.

Trả lời:

(b) Mô tả kiến trúc quản lý khóa

Với 10.000+ nhân viên, chúng ta cần một mô hình có khả năng mở rộng để xác thực khóa công khai, tránh các cuộc tấn công MITM.

1. Khởi tạo lòng tin (Bootstrap Trust): Chúng ta sẽ triển khai mô hình Certificate Authority (CA) nội bộ, tương tự như mô hình TLS/HTTPS. Tổ chức sẽ tạo ra một cặp khóa CA gốc. Khóa công khai của CA gốc này sẽ được cài đặt sẵn và được tin tưởng trên tất cả 10.000+ thiết bị đầu cuối của nhân viên.

2. Phân phối khóa (Key Distribution):

1. Mỗi nhân viên tạo cặp khóa chữ ký Ed25519 của riêng mình (khóa riêng và khóa công khai).
2. Nhân viên gửi khóa công khai của mình đến CA nội bộ để được ký.
3. CA nội bộ xác minh danh tính nhân viên và cấp phát một “chứng chỉ” (certificate) chứa khóa công khai của nhân viên đó, được ký bởi khóa riêng của CA.

4. Khi hai nhân viên muốn liên lạc, họ trao đổi các chứng chỉ này. Mỗi bên xác minh chữ ký trên chứng chỉ của người kia bằng khóa công khai CA mà họ đã tin tưởng. Điều này cung cấp tính xác thực (Authentication) và ngăn chặn tấn công MITM.

3. Luân chuyển khóa (Key Rotation): Các tài liệu được cung cấp (1.7 và 1.8) không thảo luận về các quy trình hoặc chính sách luân chuyển khóa.

Trả lời:

(c) Phân tích tính bảo mật của hệ thống

1. Nghe lén mạng (Network Eavesdropping): Hệ thống có khả năng chống nghe lén cao. Kẻ nghe lén (Eve) chỉ thấy các khóa công khai (ví dụ: $A = a \cdot G$ và $B = b \cdot G$) được trao đổi. Việc tính toán khóa bí mật chung ($S = ab \cdot G$) từ các giá trị này đòi hỏi phải giải quyết Bài toán Logarithm Rời rạc Đường cong Elliptic (ECDLP), được cho là “cực kỳ khó về mặt tính toán”.

2. Máy chủ bị xâm phạm (Server Compromise):

- **Máy chủ chuyển tiếp tin nhắn:** Nếu máy chủ chuyển tiếp tin nhắn bị xâm phạm, kẻ tấn công vẫn không thể đọc được nội dung tin nhắn. Giao thức ECDH đảm bảo rằng khóa bí mật chung S chỉ tồn tại trên thiết bị của hai người dùng cuối. Điều này duy trì tính bảo mật (Confidentiality).
- **Máy chủ CA:** Đây là một kịch bản nghiêm trọng. Nếu máy chủ CA nội bộ bị xâm phạm, kẻ tấn công có thể sử dụng khóa riêng của CA để ký các chứng chỉ giả mạo, cho phép chúng thực hiện các cuộc tấn công Man-in-the-Middle (MITM) hiệu quả và mạo danh bất kỳ nhân viên nào.

3. Thiết bị đầu cuối bị xâm phạm (Endpoint Compromise): Đây là điểm yếu nhất của hệ thống. Nếu thiết bị của nhân viên bị xâm phạm, kẻ tấn công sẽ có toàn quyền truy cập vào khóa riêng (private key) Ed25519 của nhân viên đó. Chúng có thể mạo danh nhân viên (phá vỡ tính xác thực), ký các giao dịch lừa đảo (phá vỡ tính chống chối bỏ), và giải mã tất cả các tin nhắn trong quá khứ và tương lai của họ.

4. Tấn công từ nội bộ (Insider Attacks):

- **Nhân viên độc hại:** Hệ thống mật mã đảm bảo tính xác thực, nghĩa là một nhân viên độc hại không thể mạo danh một nhân viên khác (trừ khi họ xâm phạm được CA). Tuy nhiên, hệ thống không thể ngăn cản một nhân viên đã được xác thực sử dụng quyền truy cập hợp pháp của mình để làm rò rỉ thông tin.
- **Quản trị viên CA độc hại:** Như đã đề cập ở trên, một quản trị viên có quyền truy cập vào khóa riêng của CA có thể phá vỡ toàn bộ mô hình tin cậy và thực hiện các cuộc tấn công MITM.